自动驾驶技术实践文档

nuScenes 数据集 和 视觉 YOLO 检测

目录

1. 概述 4
实践目的 4
1.1. 所需软件 4
2. 环境配置及必要软件安装 4
2.1. 安装编译器（VScode） 4
2.2. 免费 GPU 资源（Kaggle Notebook） 6
2.2.1. 简介 6
2.2.2. Kaggle Notebook 的创建与基本使用 6
2.2.3. VS Code tunnel 连接 8
2.2.4. 数据集上传 12
2.3. 本地运行-检查本机 GPU 环境 13
2.3.1. 使用任务管理器查看 GPU 13
2.3.2. 使用 nvidia-smi 查看 NVIDIA GPU 14
2.4. 安装 cuda 和 cudnn 15
2.4.1. 安装 cuda 15
2.4.2. 安装 cudnn 16
2.5. 安装 Anaconda3（可替换为更轻量的 Miniconda） 16
2.5.1. 下载安装 16
2.5.2. 验证安装 17
2.5.3. 编辑系统环境变量 17
2.6. 创建虚拟环境 18
2.7. 安装 pytorch 18
2.7.1. GPU 版本 pytorch 18
2.7.2. CPU 版本 pytorch 18
2.7.3. 验证安装 19
3. nuScenes 数据集介绍与使用 19
3.1. 概述 19
3.2. 数据收集 19
3.2.1. 场景规划 19
3.2.2. 数据采集车辆及传感器 20
3.2.3. 传感器标定 20

3.2.4. 传感器同步 21
3.3. 数据集格式 21
3.4. 数据集标注 22
3.5. 数据集下载 23
3.6. nuscenes-devkit 使用 24
3.6.1. nuScenes-devkit 安装 24
3.6.2. 数据集初始化与基本读取 24
3.6.3. 标定、车辆位姿与坐标转换 25
3.6.4. 投影与可视化相关接口 27
3.7. 2D 目标检测标签转换 28
3.7.1. 转换原理 28
3.7.2. 目标类别 29
3.7.3. 数据集划分与转换脚本 29
3.7.4. 目标筛选与关键参数 31
4. YOLO 配置及使用示例 32
4.1. 训练模型—免费 GPU 资源 32
4.2. 模型测试—免费 GPU 资源 34
4.3. 训练模型—本地 35
4.4. 模型预测—本地 37

1. 概述

实践目的

本实践旨在掌握自动驾驶感知领域中视觉检测、激光雷达 BEV 感知及端到端自动驾驶模型的核心原理与落地应用，具体包括：熟练掌握 nuScenes 自动驾驶数据集的获取、解析与预处理流程，学会免费 GPU 资源的配置、调用与实操运用，搭建完整适配的深度学习训练推理环境；完成 YOLO 视觉目标检测模型的配置、训练、调优与实测预测，掌握二维视觉目标检测的完整实操流程；选取轻量化 LiDAR BEV 感知模型开展上手实践，理解激光雷达鸟瞰图感知的核心逻辑与轻量化模型的部署优势，完成模型训练与感知验证；同时学习端到端自动驾驶模型的架构原理与运行机制，整合视觉、激光雷达多感知技术，全方位掌握自动驾驶多模态感知的技术体系与实操方法，最终实现基于多模型、多传感器的自动驾驶感知综合实践落地，夯实自动驾驶深度学习应用能力。

1.1. 所需软件

环境管理工具：Anaconda3（用于创建多 Python 版本虚拟环境，避免库版本冲突，如 YOLO 需 Python≥3.9、LabelImg 需 Python≤3.8）；

GPU 加速工具：CUDA（NVIDIA 显卡专属机械学习驱动）、cuDNN（CUDA 配套加速工具包，选装，可提升训练速度）；

深度学习库：PyTorch（核心库，GPU 版本需匹配 CUDA 版本，CPU 版本训练较慢）；

编译器：VSCode

2. 环境配置及必要软件安装

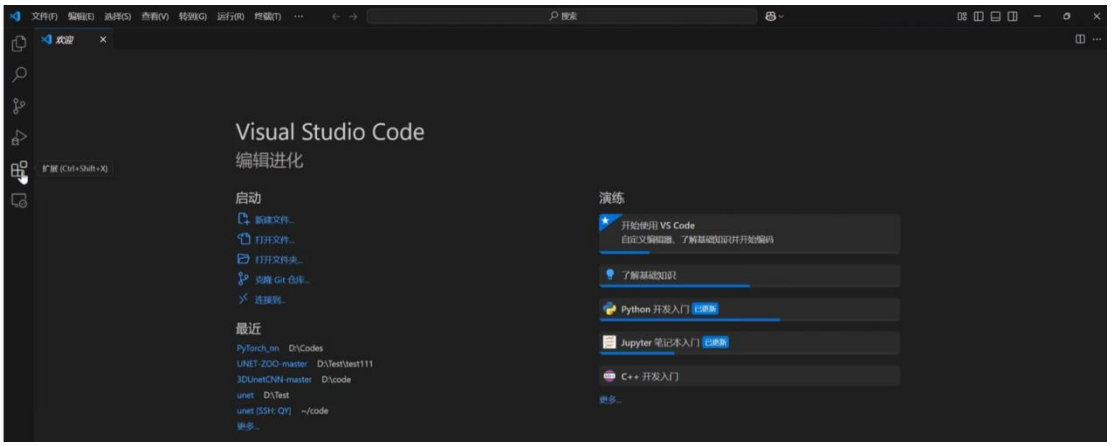
2.1. 安装编译器（VScode）

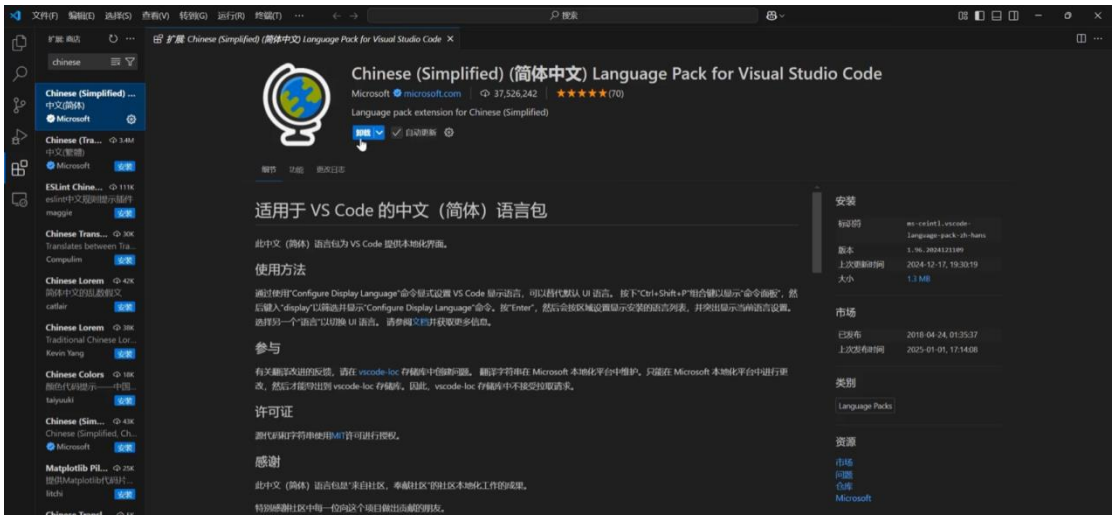
可以直接在微软的应用商店下载

也可以去官方网站下载：https://code.visualstudio.com/，官网下载时注意选择创建桌面快捷方式

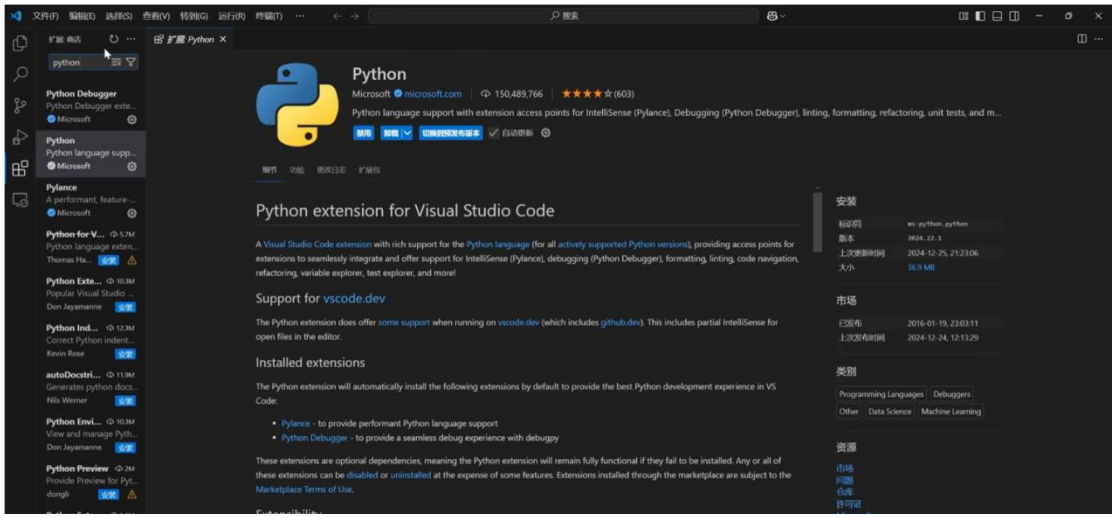


安装完成后打开 Vscode，在扩展中搜索 Chinese，安装中文扩展包，然后按照它提供的使用说明切换语言

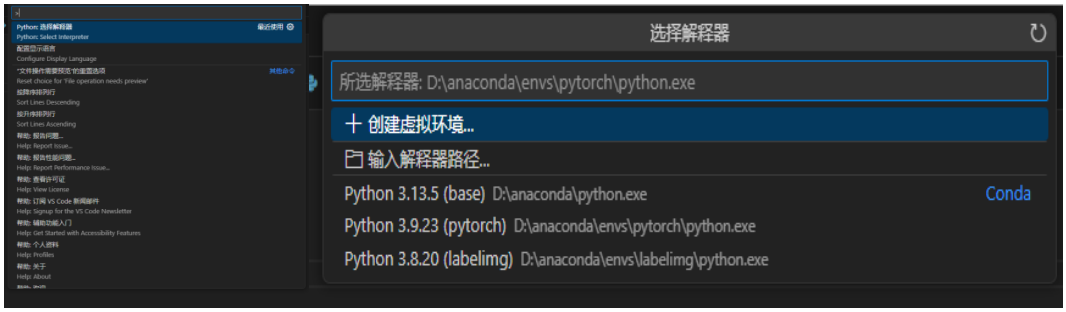




在扩展中搜索 python，安装 python 插件



同时按 Shift+Ctrl+P，找到 Python：选择解释器，点击即可在这里找到我们之前创建的虚拟环境，可以点击切换



2.2. 免费 GPU 资源（Kaggle Notebook）

2.2.1. 简介

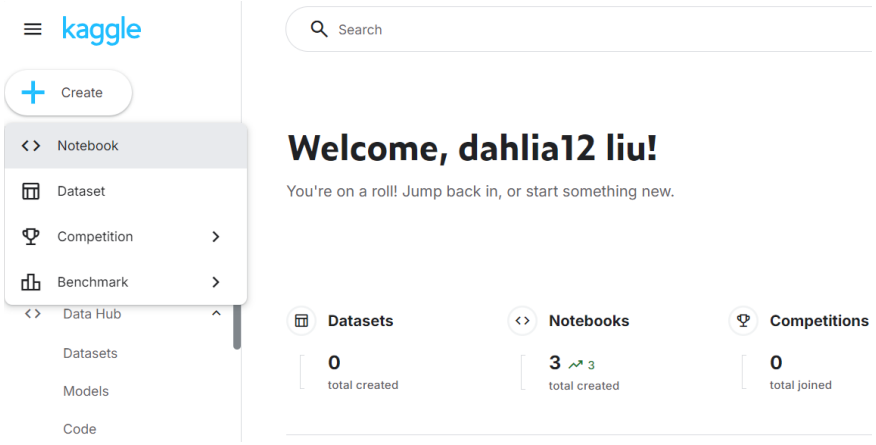
在深度学习实验中，模型训练和推理通常需要使用 GPU。对于没有 NVIDIA 独立显卡，或者本地显卡性能不足的同学，可以使用在线 GPU 平台完成课程实验。本课程推荐使用 Kaggle Notebook 作为免费的 GPU 实验环境。

Kaggle 是一个面向数据科学和机器学习的在线平台，提供数据集、机器学习竞赛以及在线编程环境等功能。其中，Kaggle Notebook 可以理解为一台已经配置好 Python 和深度学习环境的云端计算机。用户只需要通过浏览器即可编写和运行 Python 程序，不需要在自己的电脑上安装 CUDA、PyTorch 等复杂的 GPU 环境。

Kaggle Notebook 的使用页面为：https://www.kaggle.com/work。Kaggle 提供的是具有使用额度限制的免费云端 GPU，并不是一台可以长期占用的个人 GPU 服务器。因此，在完成课程实验后，应及时关闭 GPU Session，避免不必要地消耗免费 GPU 使用额度。在后续章节中，我们将进一步介绍如何创建 Kaggle Notebook、开启 GPU，以及如何通过 VS Code 连接 Kaggle Session 环境进行代码编写和实验。

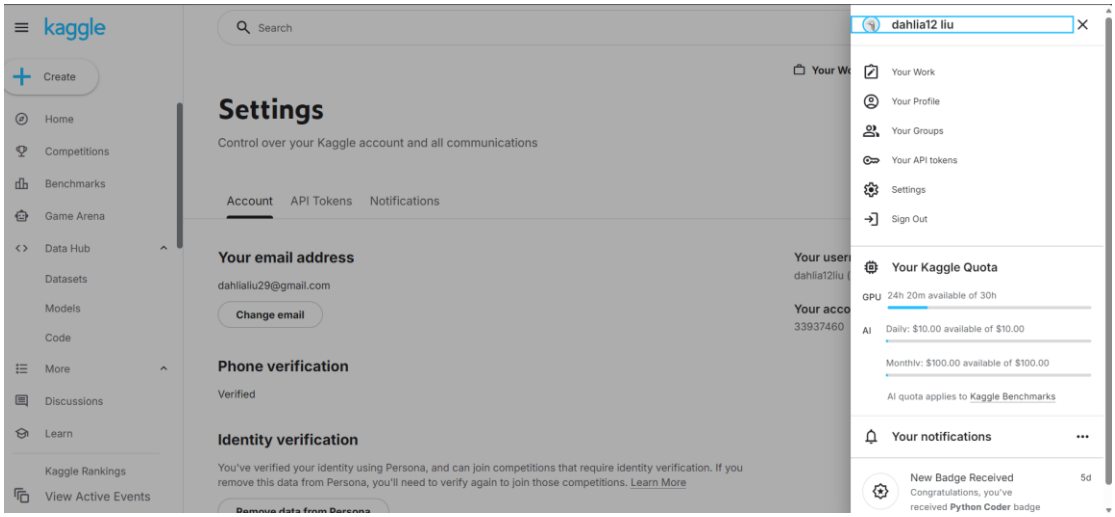
2.2.2. Kaggle Notebook 的创建与基本使用

首次使用 Kaggle 时，需要先注册 Kaggle 账号。可以使用常用邮箱进行注册，例如 QQ 邮箱，也可以直接使用 Google 账号登录。登录后进入 Kaggle 的 Your Work 页面，点击 Create → New Notebook 即可创建一个新的 Notebook。Kaggle Notebook 默认提供 Python 运行环境，创建完成后即可直接编写和运行代码，无需提前在本地安装 Python、CUDA 或 PyTorch。Notebook 左上角显示当前 Notebook 的名称，可以根据实验内容进行修改；代码以 Cell 的形式组织，可以运行单个 Cell，也可以依次运行整个 Notebook。Kaggle 官方文档也将 Notebook 定义为由代码 Cell 和 Markdown Cell 组成的在线计算环境。



如果需要使用 Kaggle 提供的免 GPU，新注册账号通常需要先完成手机号码验证（Phone Verification）。可以进入 Kaggle 账号的 Settings 页面找到 Phone Verification，按照页面提

示输入手机号并完成验证码认证。完成认证后，在 Notebook 的 Settings 中即可选择 Accelerator，将默认的 CPU 环境切换为可用的 NVIDIA GPU。Kaggle 官方说明，GPU Accelerator 的使用需要账号完成手机验证；如果新账号中看不到 GPU 选项，也应首先检查 Phone Verification 是否已经完成。



创建 Notebook 后，其主要功能如图所示。页面中间为代码编辑区域，顶部的运行按钮可以运行当前 Cell 或运行多个 Cell；右上角显示当前 Session 的运行状态，并可以停止或重新启动 Session。右侧的 Input 用于添加课程数据集、Kaggle Dataset 或其他 Notebook 的输出数据；Output 则用于查看程序运行产生的文件。程序默认的工作目录为/kaggle/working/，例如模型权重、训练日志和预测结果可以保存到该目录。需要注意，/kaggle/working/是程序的工作和输出目录，而不是 Notebook 源代码文件实际所在的位置；Notebook 本身由 Kaggle 平台管理。Kaggle 当前允许 Notebook 在/kaggle/working/中保存最多约 20 GB 的输出数据。

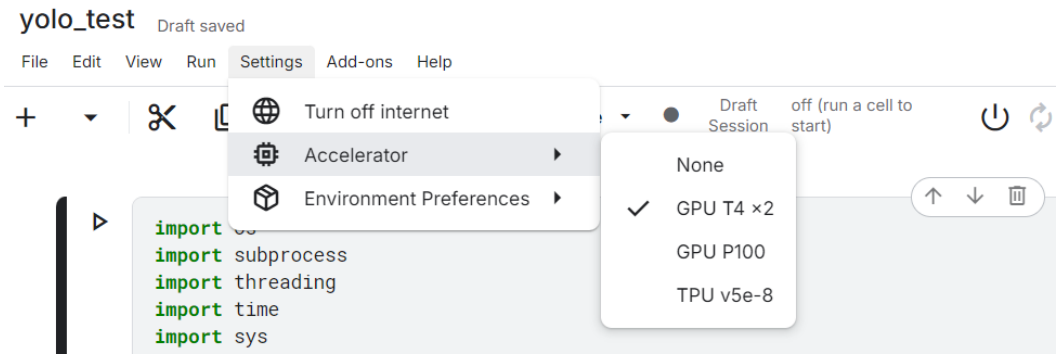


Kaggle Notebook 可以理解为保存在云端的实验文件，而 Session 是真正负责执行这些代码的临时云端计算环境。启动 Session 后，Kaggle 会为当前 Notebook 分配 CPU、GPU、内存、显存和临时磁盘，并运行 Python、PyTorch 等程序。因此，Notebook 更像“实验代码本身”，Session 更像“临时分配的一台云端电脑”。即使关闭网页，Notebook 中已保存的代码仍然存在，但当前 Session 可能仍继续运行并占用 GPU 额度，所以实验结束后应主动停止 Session。

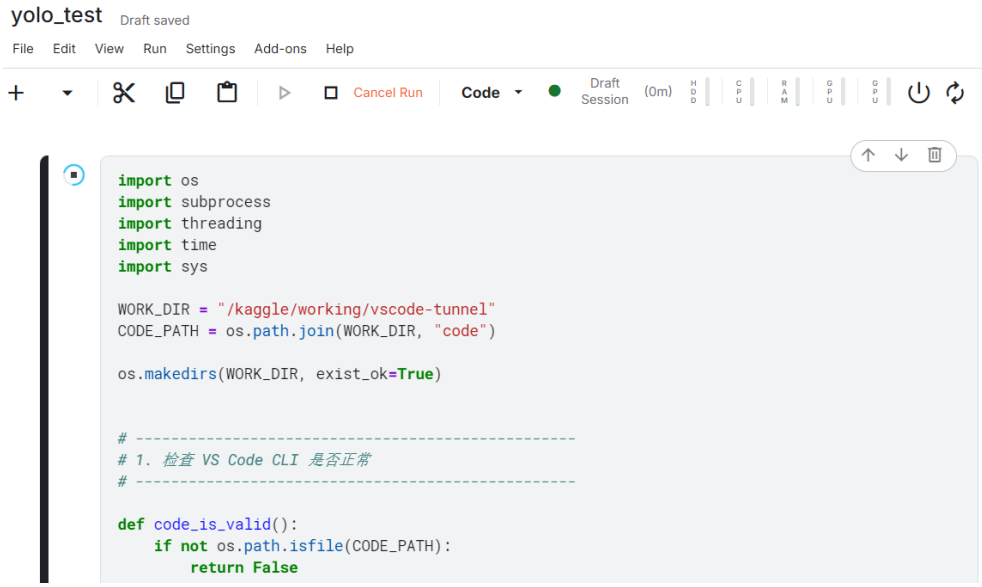
为了避免 Session 关闭或重新启动后/kaggle/working/中的文件消失，可以在右侧 Session options→Persistence 中选择 Files only。开启后，Kaggl 会尝试在不同的交互式 Session 之间保留/kaggle/working/中的文件，因此例如已经下载的数据、训练得到的权重或其他中间结果，在重新启动 Session 后通常仍可以继续使用。如果选择 No persistence，重新启动 Session 时则会重新获得一个较为干净的运行环境。需要注意，Kaggle 将 Persistence 定义为 best-effort 机制，例如 Session 异常崩溃时仍可能发生数据丢失，因此重要的模型和实验结果还应通过 Save Version 保存为 Notebook 的正式输出，或另外备份，而不能只依赖 Session Persistence。

2.2.3. VS Code tunnel 连接

首先打开需要使用的 Kaggle Notebook，在右侧 Session options 中开启 Internet，并在 Accelerator 中选择当前可用的 GPU。Remote Tunnel 需要通过互联网连接 VS Code 服务，因此 Internet 必须保持开启；同时启动 Notebook 的 Session。



在 Kaggle Notebook 中新建一个 Code Cell，下载并解压 VS Code CLI，下载脚本参考 https://pan.quark.cn/s/a9059fdcdb32，将代码复制在 Cell 中执行，其中 kaggle-gpu-node 是远程计算机的名称，可以自行修改。VS Code CLI 用于在 Kaggle 云端启动 Remote Tunnel。



执行结果如下图。在浏览器中打开 github.com/login/device，登录自己的 GitHub 账号，并输入 Kaggle 输出的 8 位验证码（图中 505E-C39E）完成授权。授权成功后，Kaggle 中会显示 Tunnel 已建立。后续本地 VS Code 也需要使用同一个 GitHub 账号登录。

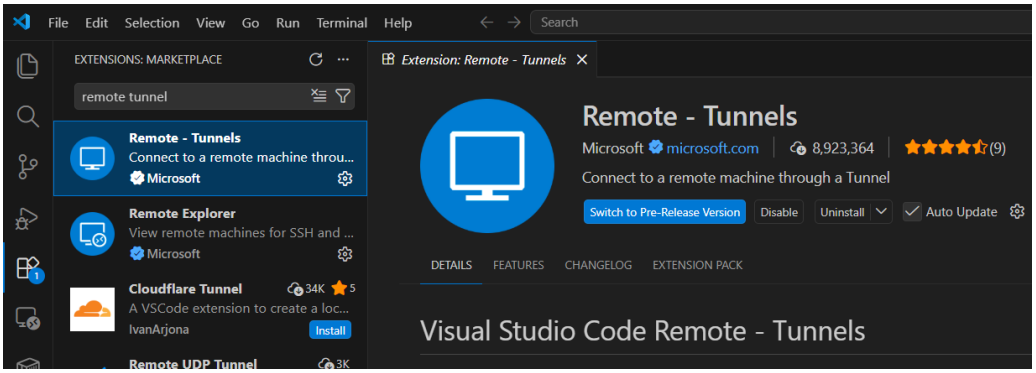
```
正在下载 VS Code CLI...
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   162  100   162    0     0    289      0 --:--:-- --:--:-- --:--:--   289
100 12.1M  100 12.1M    0     0  12.5M      0 --:--:-- --:--:-- --:--:-- 64.6M
code 1.135.0 (commit 08d4889f9ec4a1685d257b9b95de036c8e1ce1e5)
VS Code CLI 下载完成。

[Heartbeat] VS Code Tunnel 仍在运行 2026-09-01 16:49:27

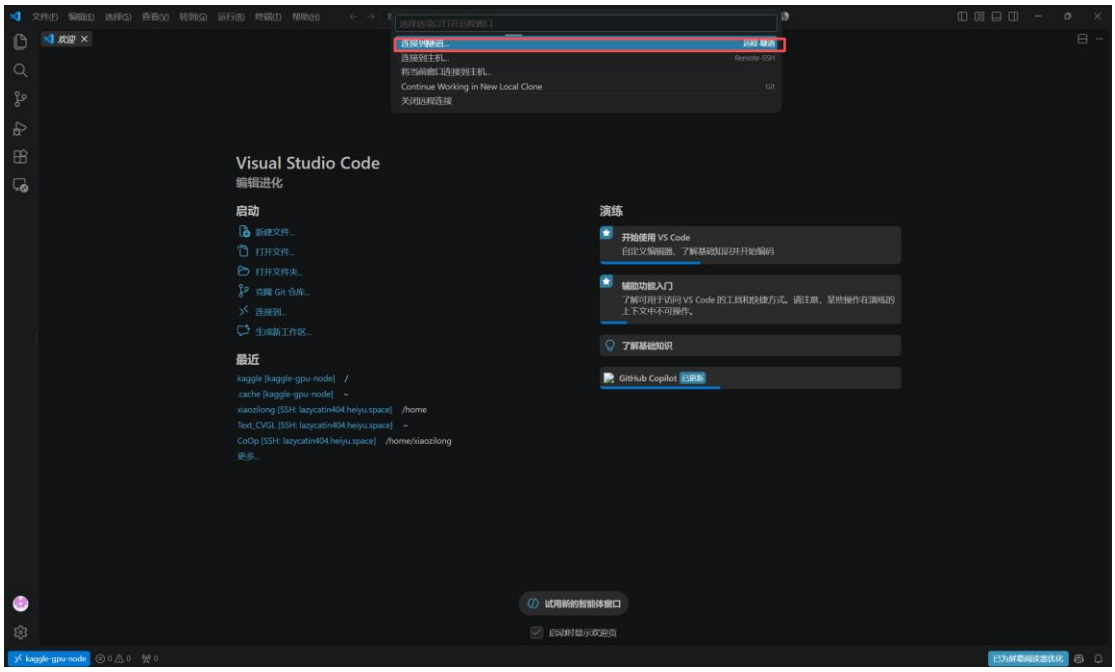
正在启动 VS Code Tunnel...
首次运行请根据输出完成 GitHub/Microsoft 登录授权。

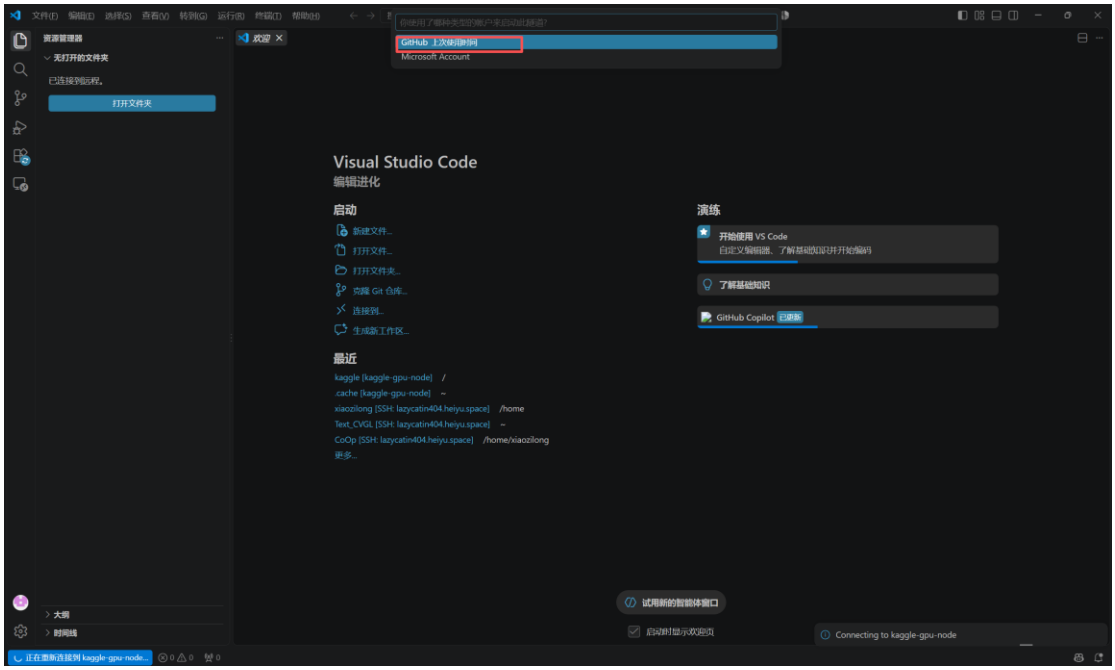
*
* Visual Studio Code Server
*
* By using the software, you agree to
* the Visual Studio Code Server License Terms (https://aka.ms/vscode-server-license) and
* the Microsoft Privacy Statement (https://privacy.microsoft.com/en-US/privacystatement).
*
[2026-09-01 16:49:28] info Using GitHub for authentication, run `code tunnel user login --provider <prov
ider>` option to change this.
To grant access to the server, please log into https://github.com/login/device and use code 505E-C39E
```

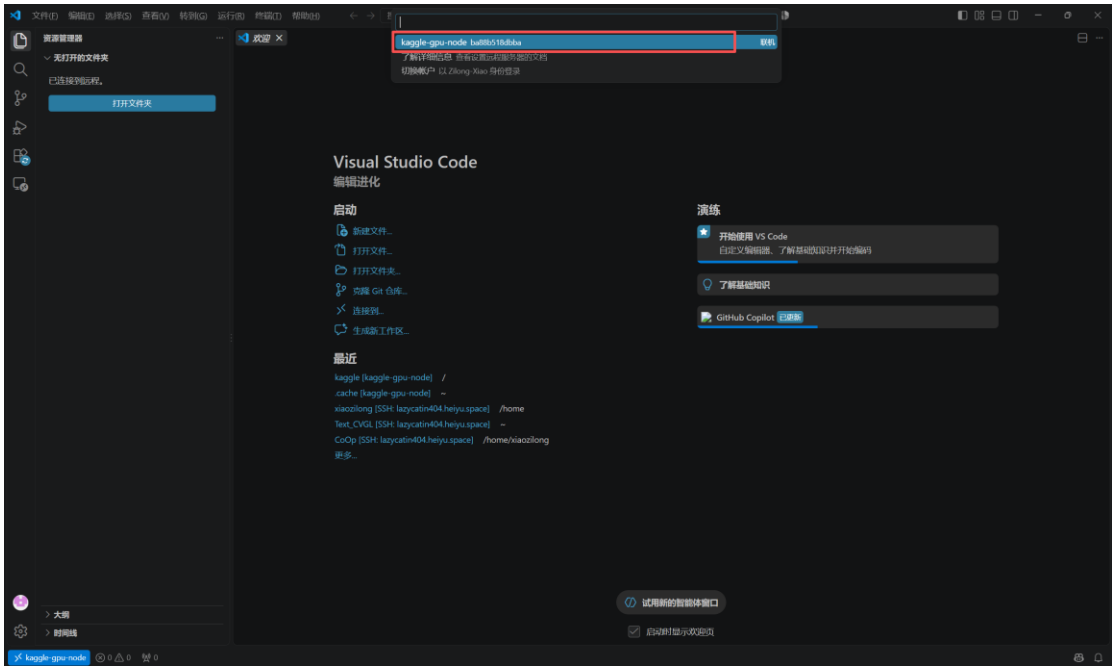
在本地电脑打开 VS Code，在扩展商店安装微软提供的 Remote - Tunnels 扩展，并登录刚才用于授权的 GitHub 账号。



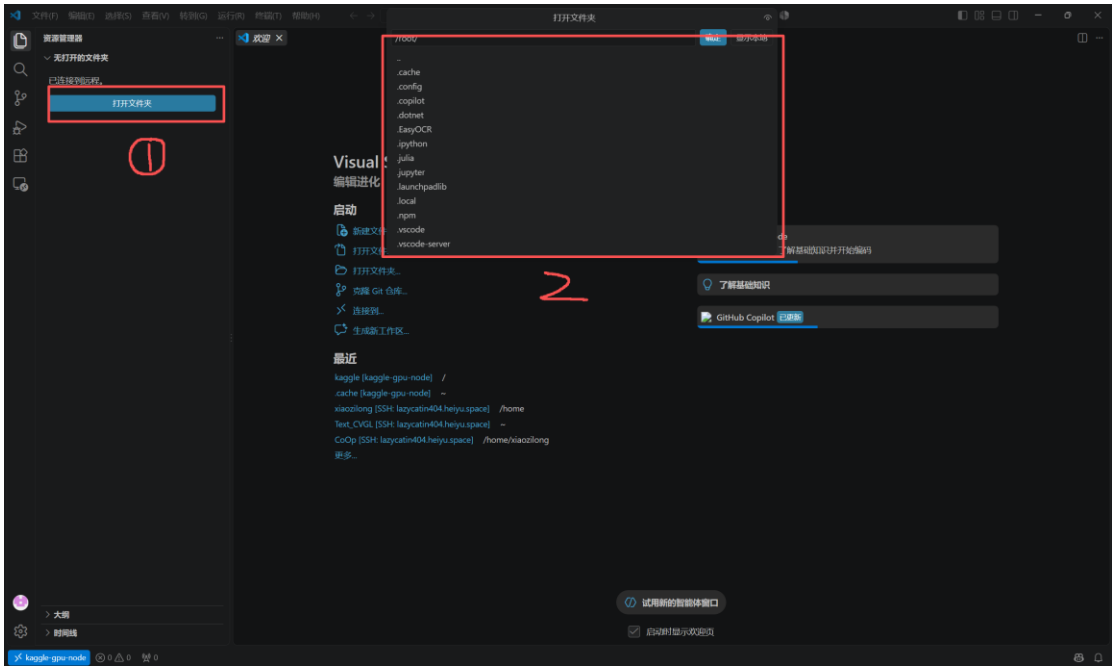
点击 VS Cod 左下角的远程连接按钮，选择 Connect to Tunnel，选择 GitHub，然后在设备列表中选择前面创建的 kaggle-gpu-node。

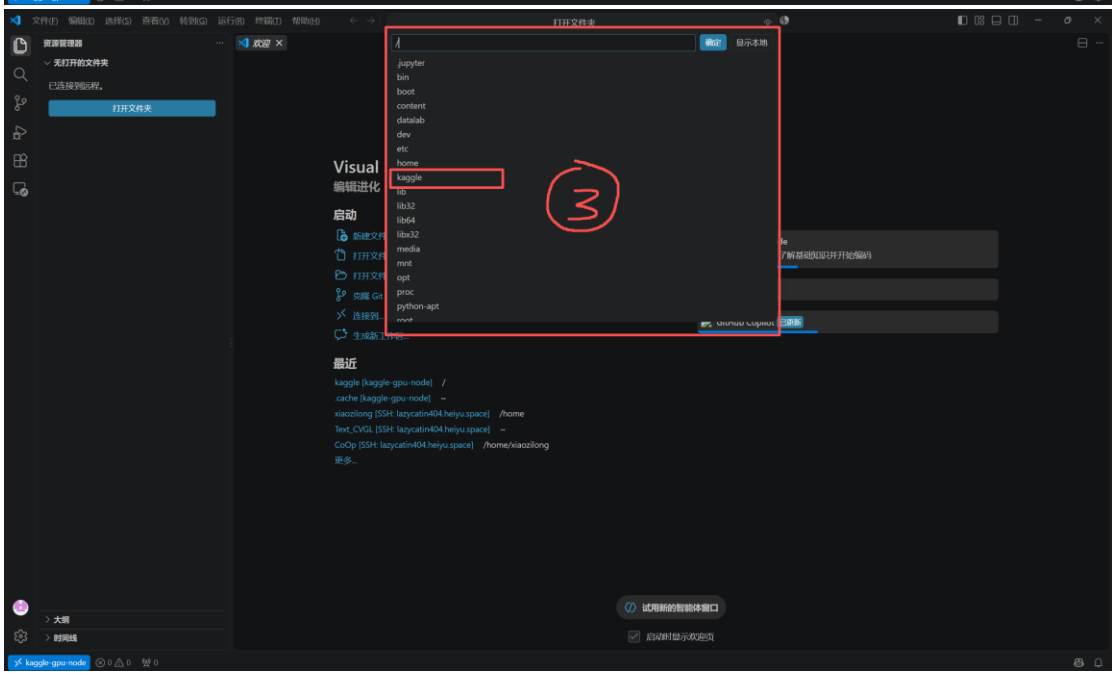


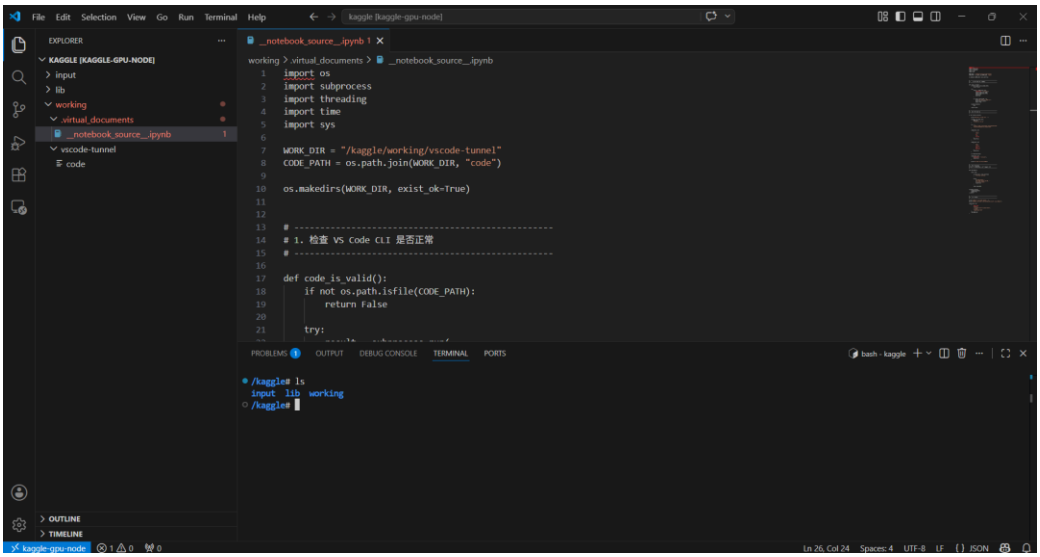




连接成功后，本地 VS Code 实际执行代码所使用的是 Kaggle 的远程计算资源，默认目录在 root 下，要回到根目录找到 kaggle 文件夹。


打开文件夹
打开文件夹
1
/root/
.cache
.config
.copilot
.dotnet
.EasyOCR
.ipython
.julia
.jupyter
.launchpadlib
.local
.npm
.vscode
.vscode-server
2
了解基础知识
GitHub Copilot
最近
kaggle [kaggle-gpu-node] /
.cache [kaggle-gpu-node] ~
xiaozilong [SSH: lazycatin404.heiyu.space] /home
Text_CVGL [SSH: lazycatin404.heiyu.space] ~
CoOp [SSH: lazycatin404.heiyu.space] /home/xiaozilong
更多...
试用新的智能体窗口
启动时显示欢迎页


打开文件夹
/
确定
显示本地
.jupyter
bin
boot
content
datalab
dev
etc
home
kaggle
lib
lib32
lib64
libx32
media
mnt
opt
proc
python-apt
3
最近
kaggle [kaggle-gpu-node] /
.cache [kaggle-gpu-node] ~
xiaozilong [SSH: lazycatin404.heiyu.space] /home
Text_CVGL [SSH: lazycatin404.heiyu.space] ~
CoOp [SSH: lazycatin404.heiyu.space] /home/xiaozilong
更多...
试用新的智能体窗口
启动时显示欢迎页




```
To grant access to the server, please log into https://github.com/login/device and use code 5905-5FDE
[2026-09-01 17:07:33] info Creating tunnel with the name: kaggle-gpu-node
[2026-09-01 17:07:34] info Adopting existing tunnel (ID=ID { cluster: "jpe1", id: "spiffy-lake-bfqz008" })

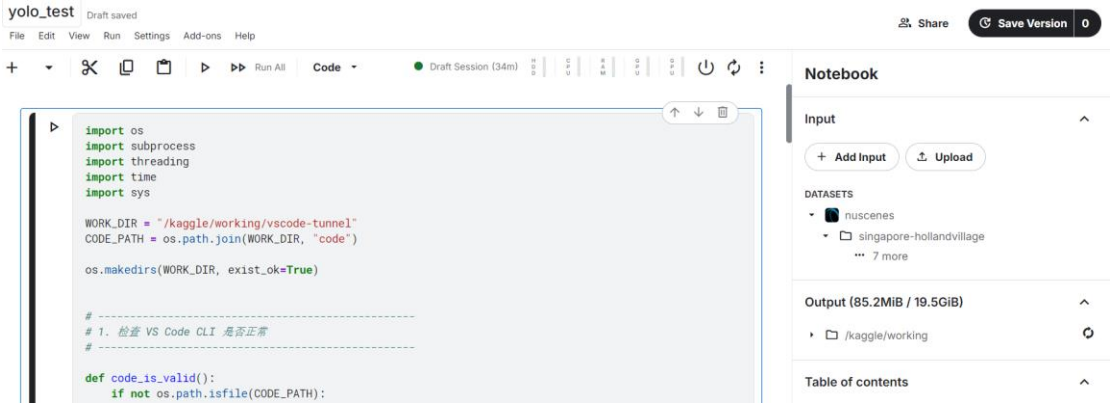
  Visual Studio Code Tunnel v1.135.0

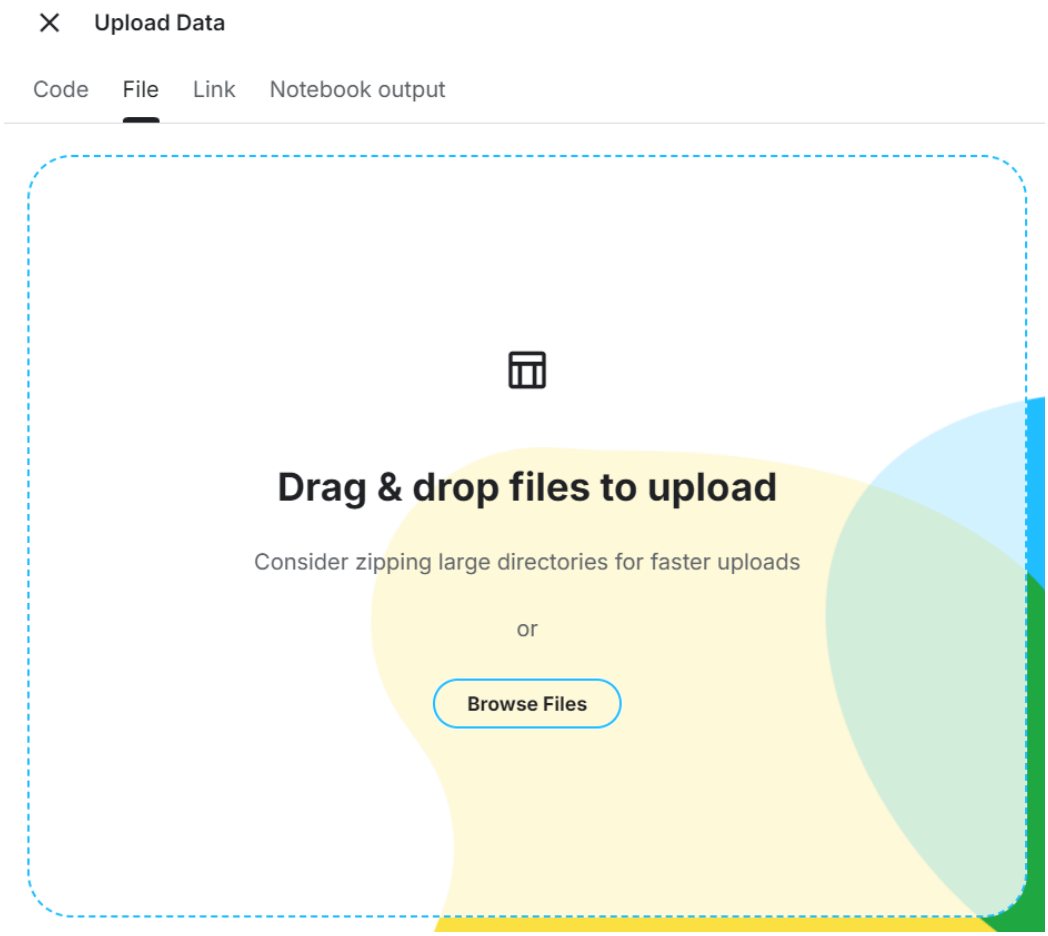
  ➜  Tunnel:   kaggle-gpu-node
  ➜  Open:  https://vscode.dev/tunnel/kaggle-gpu-node/kaggle/working/vscode-tunnel

[2026-09-01 17:08:14] info [tunnels::connections::relay_tunnel_host] Opened new client on channel 2
[2026-09-01 17:08:14] info [russh::server] wrote id
[2026-09-01 17:08:15] info [russh::server] read other id
[2026-09-01 17:08:15] info [russh::server] session is running
[2026-09-01 17:08:18] info No agent host supervisor running; starting one in the background
[2026-09-01 17:09:17] info [rpc.0] Checking /root/.vscode/cli/servers/Stable-8b640eef5a6c6089c029249d48efa5c99adf
7d51/log.txt and /root/.vscode/cli/servers/Stable-8b640eef5a6c6089c029249d48efa5c99adf7d51/pid.txt for a running
server...
[2026-09-01 17:09:18] info [rpc.0] Downloading Visual Studio Code server -> /tmp/.tmp334swc/vscode-server-linux-x
64.tar.gz
[2026-09-01 17:09:26] info [rpc.0] Starting server...
[2026-09-01 17:09:26] info [rpc.0] Server started
```

2. 2. 4. 数据集上传

点击右侧 upload 上传自定义数据集，上传后的数据集在路径/kaggle/input 下，在每次重启 Session 时会重新加载数据集，先用小数据集测试。





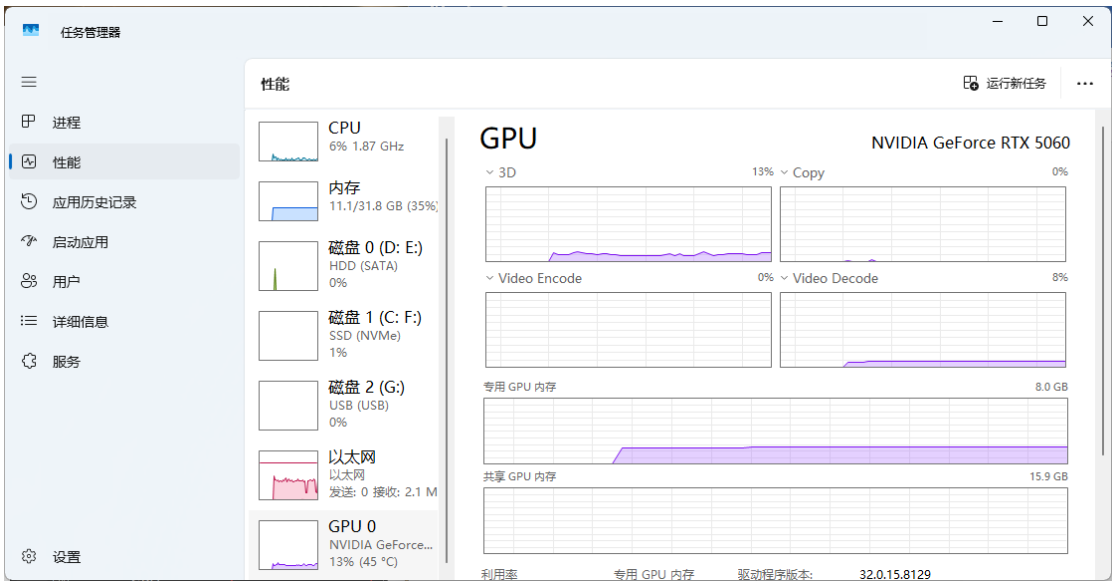
2.3. 本地运行-检查本机 GPU 环境

深度学习模型的训练通常需要进行大量矩阵运算，因此使用 GPU 可以显著提高训练速度。在本课程的实验中，推荐使用支持 CUDA 的 NVIDIA 独立显卡进行模型训练。在安装 PyTorch 等深度学习环境之前，首先需要确认计算机是否配备可用的 NVIDIA GPU，并了解 GPU 的具体型号。可以通过以下两种方法进行检查。

2.3.1. 使用任务管理器查看 GPU

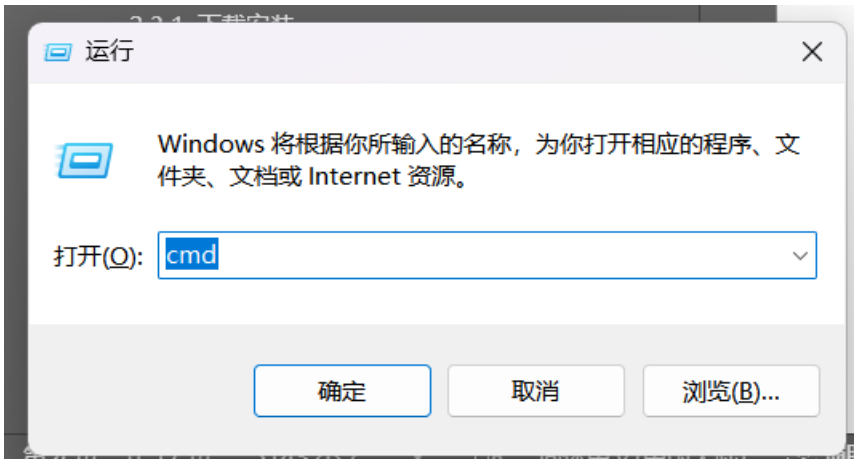
在 Windows 系统中，可以通过任务管理器快速查看计算机的显卡信息。具体步骤如下：按下 Ctrl + Shift + Esc 打开任务管理器；点击左侧的 性能（Performance）；在设备列表中找到 GPU；点击对应的 GPU，即可查看显卡型号、显存占用以及 GPU 使用率等信息。

如果计算机中存在多个 GPU，任务管理器可能会显示 GPU 0、GPU 1 等多个设备。例如，笔记本电脑通常同时具有集成显卡和 NVIDIA 独立显卡。在 GPU 页面右上角可以查看显卡的具体型号。例如，下图所示计算机配置了 NVIDIA GeForce RTX 5060 显卡，说明该计算机具有 NVIDIA 独立 GPU，可以安装支持 CUDA 的 PyTorch 版本。



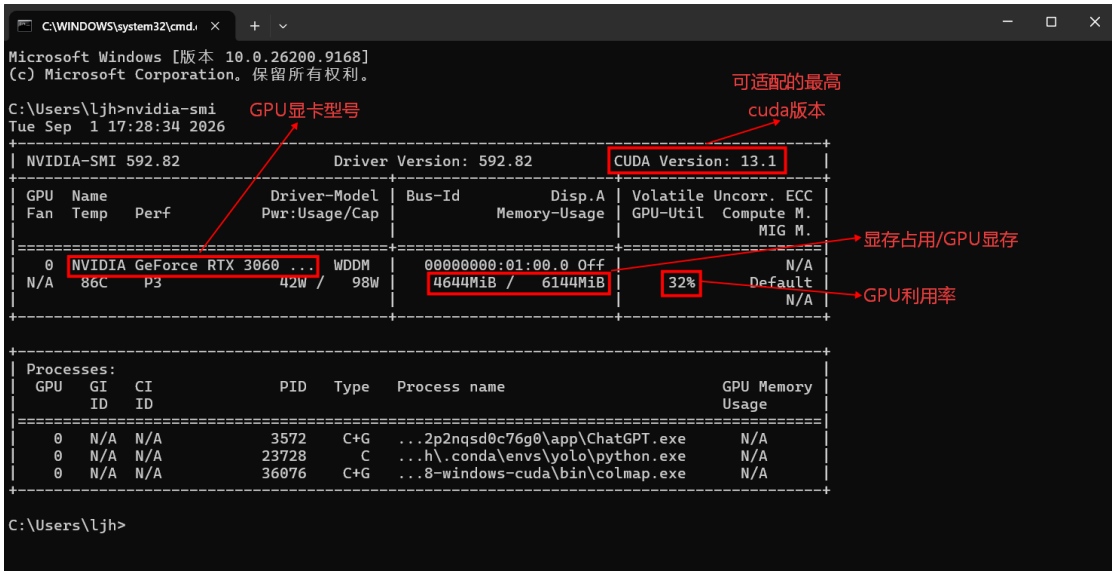
2.3.2. 使用 nvidia-smi 查看 NVIDIA GPU

如果计算机安装了 NVIDIA 显卡驱动，还可以通过 nvidia-smi 命令进一步查看 GPU 和驱动信息。首先按下：Win + R；在弹出的“运行”窗口中输入：cmd；然后按 Enter 打开 Windows 命令提示符；在命令行中输入：nvidia-smi。



如果 NVIDIA 显卡驱动安装正常，将会显示类似下图的信息，其中通常包括：GPU 型号；NVIDIA 驱动版本（Driver Version）；CUDA Driver 支持的最高 CUDA 版本；GPU 显存总量及当前显存占用；GPU 利用率；当前正在使用 GPU 的进程。例如，在输出中的 GPU Name 一栏可以查看具体的 NVIDIA GPU 型号。

如果输入 nvidia-smi 后能够正常显示 GPU 信息，说明 NVIDIA 显卡驱动已经安装，并且系统能够正确识别 NVIDIA GPU。如果出现：'nvidia-smi' 不是内部或外部命令，或者无法检测到 NVIDIA GPU，则可能存在以下情况：计算机没有安装 NVIDIA 独立显卡；NVIDIA 显卡驱动尚未安装；NVIDIA 显卡驱动安装异常。需要注意的是，nvidia-smi 中显示的 CUDA Version 表示当前显卡驱动最高支持的 CUDA 版本，并不代表计算机已经安装了对应版本的 CUDA Toolkit。

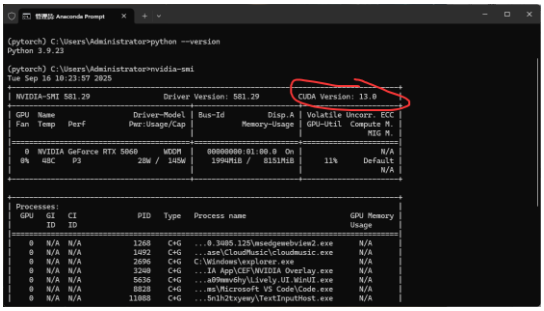


2. 4. 安装 cuda 和 cudnn

2. 4. 1. 安装 cuda

Cuda 是英伟达显卡针对机器学习所提供的显卡驱动，已安装 GeForce 驱动或者 Studio 驱动的可以不安装，直接进行 1.5.1 安装 GPU 版本的 pytorch，AMD 显卡或者没有独立显卡，直接进行 1.5.2 安装 CPU 版本的 pytorch

首先我们在 conda 终端中输入“nvidia-smi”来查看我们显卡所能支持的最高 cuda 版本，A 卡和无独立显卡则会显示非外部或内部命令



根据查询的结果前往 cuda 官网下载对应的 cuda 版本，只要是低于所能支持的最高版本都可以用，不建议下载 13.0，目前没有匹配的 cudnn 版本，cuda 官网下载地址：https://developer.nvidia.com/cuda-toolkit-archive，选择不高于你显卡所能支持的最高版本的 cuda 进行下载，安装，默认是 C 盘，不建议修改，需要预留 20G 空间，下载完成后点击文件安装，选择自定义，全部勾选



安装完成后在 conda 终端中输入“nvcc -V”，如下显示则安装成功

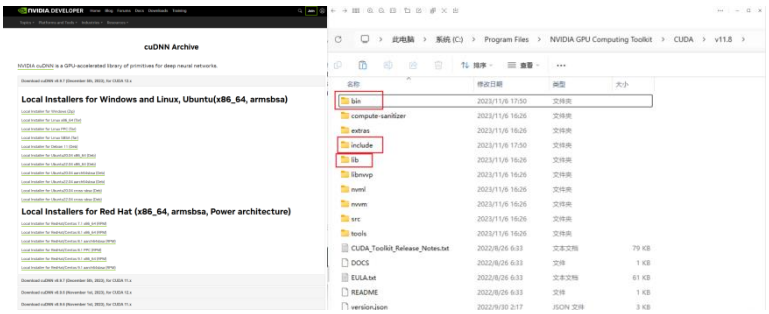
```
(pytorch) C:\Users\Administrator>nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Wed_Jul_16_20:06:48_Pacific_Daylight_Time_2025
Cuda compilation tools, release 13.0, V13.0.48
Build cuda_13.0.r13.0/compiler.36260728_0

(pytorch) C:\Users\Administrator>
```

2.4.2. 安装 cudnn

Cudnn 是 cuda 配套的深度学习加速工具包，可以不安装，安装后训练会快一些，Cudnn 官网安装地址：https://developer.nvidia.com/rdp/cudnn-archive，选择与你的 cuda 版本相匹配的 cudnn 版本，选择 Windows 压缩包，下载需要注册登录，也可以复制链接迅雷下载，下载后解压

解压后得到下图三个文件夹，全选复制进 cuda 的文件夹中进行覆盖替换，替换完成后即 cudnn 安装完成，cuda 的文件夹默认在 C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v1x.y



2.5. 安装 Anaconda3（可替换为更轻量的 Miniconda）

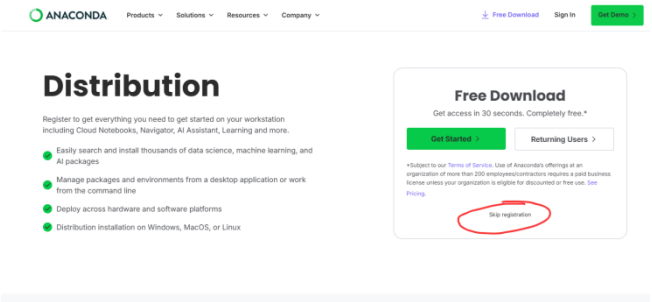
2.5.1. 下载安装

由于我们会使用多个模型以及工具，它们对于 python 版本的要求是不一样的，比如 YOLO 要求 python 版本在 3.9 及以上，而 labelimg 则要求 python 版本不能高于 3.9，否则将会出现问题，所以我们需要安装 Anaconda 来帮助我们管理多个 python 版本的环境。

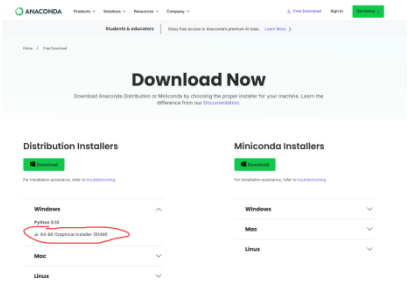
简单的说，不安装 Anaconda 则所有的库只能安装到一个环境中，python 版本也只能有一个，非常不方便，而安装 Anaconda 之后则可以拥有多个环境，把不同需求的库安装到不同环境中，使用非常方便。

Anaconda 官网下载地址：https://www.anaconda.com/download

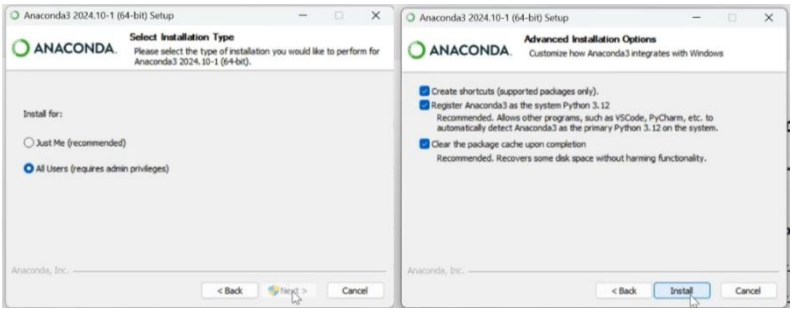
点击 skip registration 跳过注册



选择 Windows 版本的安装包，图中所示为 python 版本为 13.0 的 conda，conda 自带的 base 环境（基础环境）的 python 环境为 13.0.

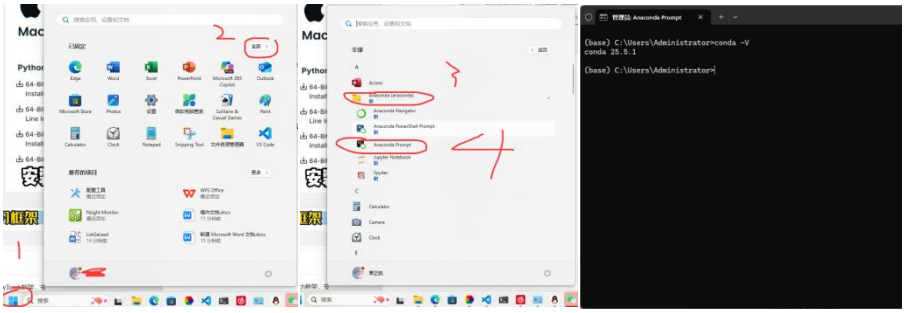


下载完成后点击下载的文件进行安装，一直点击下一步，有些界面需要进行选择，按照图示进行选择，下载路径可以自定义（不要用中文），不建议安装在 C 盘（比如 D:\Anaconda 即可），一定要记得自定义的下载路径，后续可能要用



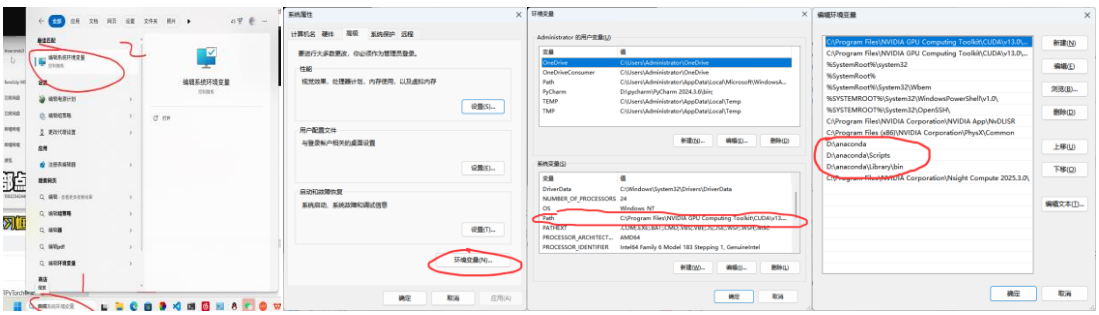
2. 5. 2. 验证安装

下载完成后在开始中找到 conda 的终端，点击运行，在终端中输入“conda -V”并回车，如下显示则安装成功，如果显示非外部或者内部命令则需要编辑环境变量



2. 5. 3. 编辑系统环境变量

在搜索中搜索编辑系统环境变量，点击环境变量，找到 Path 系统变量，将刚刚下载的 conda 文件中的这三个文件的路径复制然后新增到其中，点击确认就会关闭窗口，一定不要直接叉掉，不然就不会保存修改

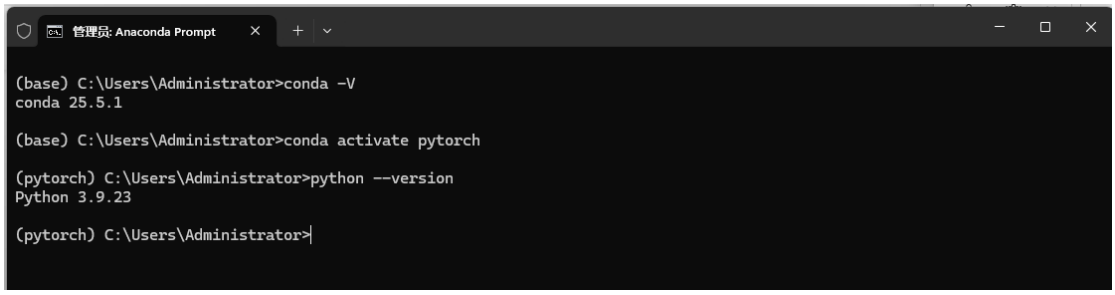


2.6. 创建虚拟环境

Conda 安装成功后即可创建虚拟环境，在 conda 终端中输入“conda create -n pytorch python=3.9”并回车，conda 就会开始创建一个名字为 pytorch 的 python 版本为 3.9（可自定义名字和 python 版本）的虚拟环境，这里输入 y 并回车



环境创建完成后，输入“conda activate pytorch”并回车，当命令行前面括号中由 base 变为 pytorch 时，则表示成功进入了 pytorch 环境，此时还可以输入“python --version”来查看当前环境的 python 版本

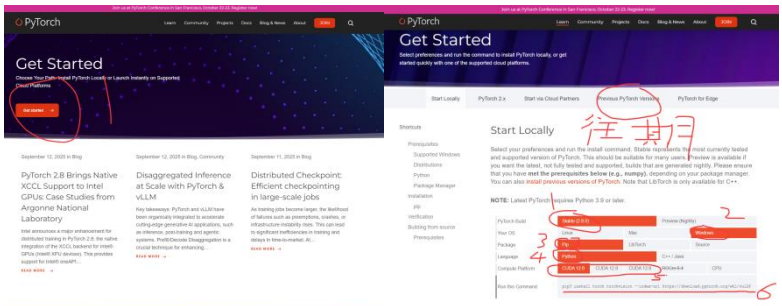


2.7. 安装 pytorch

2.7.1. GPU 版本 pytorch

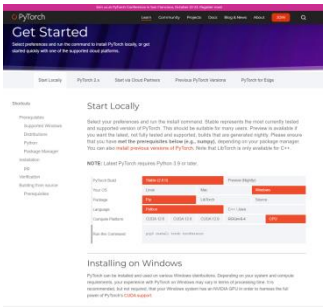
Pytorch 是 python 进行深度学习的必要库，有英伟达显卡则安装 GPU 版本的，官网地址：https://pytorch.org/

如图所示选择不高于你 cuda 版本的可通过 pip 下载的 Windowspython 稳定版，如果界面没有适合的版本可以在往期版本中找适合的，复制提供的指令输入到 conda 终端中回车运行（确保在新创建的环境中而非 base 环境），有需要输入 y 或者 n 的输入 y 即可



2.7.2. CPU 版本 pytorch

没有英伟达显卡的，直接选择 CPU 即可，同样复制运行即可，有需要输入 y 或者 n 的输入 y 即可



2. 7. 3. 验证安装

安装完成后，在 conda 终端中依次输入“python”“import torch”“torch.cuda.is_available()”并回车，每个指令之间可能需要等待一会儿，最后输出 Ture 即为安装成功

```
(pytorch) C:\Users\Administrator>python
Python 3.9.23 (main, Jun  5 2025, 13:25:08) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> torch.cuda.is_available()
True
>>>
```

3. nuScenes 数据集介绍与使用

本实践以 nuScenes 自动驾驶数据集为例，使学生了解典型自动驾驶多传感器数据集的数据组成、采集方式、标注形式和组织结构，掌握 nuScenes 开发工具包 DevKit 的基本使用方法，理解相机、LiDAR、自车坐标系和全局坐标系之间的转换关系，并能够完成 LiDAR 点云向相机图像的投影、多传感器数据可视化以及 nuScenes 到 YOLO 二维目标检测数据格式的转换。

3. 1. 概述

nuScenes 是由 Motional 团队开发并公开的大规模自动驾驶数据集，主要用于支持计算机视觉和自动驾驶领域的研究。与只提供图像或单一传感器数据的数据集不同，nuScenes 同时提供 Camera、LiDAR、Radar、车辆位姿以及地图等多种信息，因此非常适合用于学习自动驾驶中的多传感器感知与数据融合。

nuScenes 共包含 1000 个驾驶场景，数据采集地点位于美国 Boston 和新加坡。每个场景长度约为 20s，由人工从连续驾驶数据中选取，覆盖城市道路、交叉路口、车辆转向、变道、停车以及复杂交通参与者等多种驾驶情况。

这里首先需要理解 nuScenes 中的一个基本概念：scene。一个 scene 可以理解为“一段连续的驾驶片段”。例如车辆从进入一个路口、等待、转弯到驶离路口，这一段连续过程就可以构成一个 scene。

对于本课程而言，学习 nuScenes 不仅是为了了解一个自动驾驶数据集，更重要的是理解自动驾驶数据之间的关系。后续实验中将利用 nuScenes 完成 Camera 图像读取、LiDAR 点云读取、LiDAR 到 Camera 的投影以及 3D 标注到二维图像的转换。

3. 2. 数据收集

3. 2. 1. 场景规划

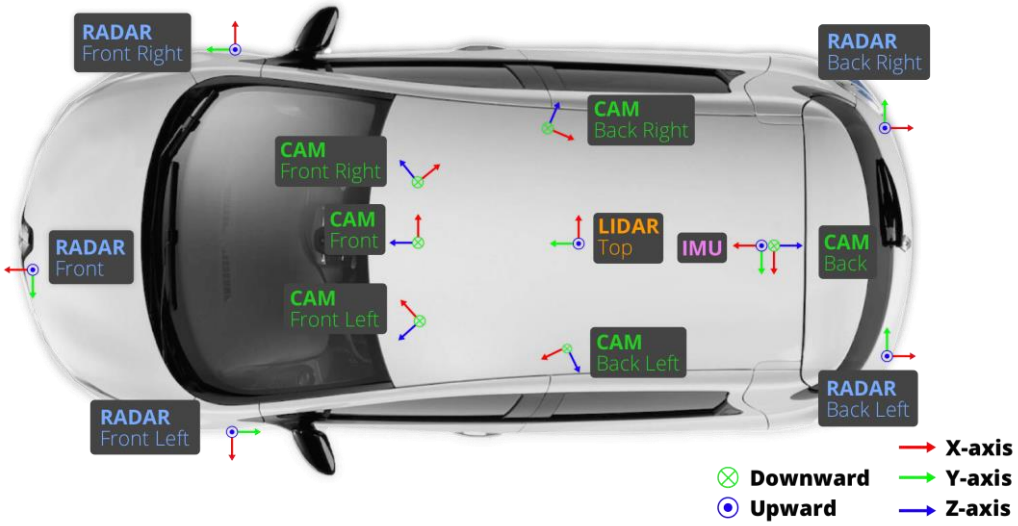
nuScenes 在 Boston 和新加坡共采集了约 15h 的真实道路驾驶数据。数据采集区域包括

Boston Seaport，以及新加坡的 One North、Queenstown 和 Holland Village 等区域。在设计采集路线时，nuScenes 并不是简单地让车辆随机行驶，而是尽可能选择具有代表性和挑战性的道路环境，使数据覆盖不同地点、时间、天气以及交通状况。例如，数据中包含城市道路、交叉路口、车辆密集区域、行人、自行车以及大型车辆等不同交通参与者。完成连续驾驶数据采集后，官方从中选出了 1000 个约 20s 的片段作为最终的 scene，并进一步对其中的关键帧进行人工标注。

3.2.2. 数据采集车辆及传感器

nuScenes 使用两辆具有相同传感器配置的 Renault Zoe 车辆，分别在 Boston 和新加坡进行数据采集。nuScenes 采集车辆搭载了 1 个 LiDAR、5 个 Radar 和 6 个 Camera，同时还包含 GPS、IMU 以及高精地图等相关信息。不同传感器能够从不同角度描述道路环境：Camera 主要提供颜色和纹理信息，LiDAR 能够获得准确的三维空间结构，Radar 则可以测量较远目标的位置和径向速度。车辆主要传感器配置如下图所示。

车辆顶部安装 1 个 Velodyne HDL32E 旋转式 LiDAR。该 LiDAR 具有 32 个激光通道，扫描频率约为 20Hz，能够获取车辆周围 360°范围内的三维点云。LiDAR 数据是自动驾驶三维目标检测和环境建模中非常重要的数据来源。车辆同时搭载 5 个毫米波 Radar，采样频率约为 13Hz。与 Camera 和 LiDAR 相比，Radar 不仅可以测量目标距离，还可以直接获取目标的径向速度，并且在较远距离以及部分恶劣天气条件下具有一定优势。车辆搭载 6 个彩色 Camera，采样频率约为 12Hz，发布图像的分辨率为 1600×900。6 个 Camera 分别位于车辆前方、前左、前右、后方、后左和后右，在数据集中分别命名为 CAM_FRONT、CAM_FRONT_LEFT、CAM_FRONT_RIGHT、CAM_BACK、CAM_BACK_LEFT 和 CAM_BACK_RIGHT。通过这些 Camera，可以获得车辆周围接近 360°的视觉信息，是后续进行二维目标检测和视觉感知的主要数据来源。除此之外，车辆还搭载 IMU 和 GPS 设备，并结合地图等信息估计车辆自身的位置和姿态。这些车辆位姿信息在多传感器坐标转换过程中非常重要。



3.2.3. 传感器标定

Camera、LiDAR 和 Radar 安装在车辆上的不同位置，并且具有不同的朝向。因此，同一个目标在不同传感器中的坐标并不相同。为了让不同传感器的数据能够相互转换，nuScenes 提供了完整的传感器标定信息。对于每一个传感器，都给出了其相对于车辆坐标系的位置和姿态；对于 Camera，还额外提供相机内参。

在 nuScenes 中，这些信息主要保存在 calibrated_sensor 表中。可以简单理解为：

calibrated_sensor 负责回答“这个传感器安装在车辆的什么位置、朝向哪里”。例如，当我们希望把 LiDAR 点云投影到前视 Camera 图像时，就必须知道 LiDAR 相对于车辆的位置和方向，以及 Camera 相对于车辆的位置和方向。对于 Camera，还需要知道焦距和主点等相机内参，才能将 Camera 坐标系中的三维点进一步投影到二维图像中。因此，传感器标定信息是后续 LiDAR-Camera 投影最重要的数据之一。

3.2.4. 传感器同步

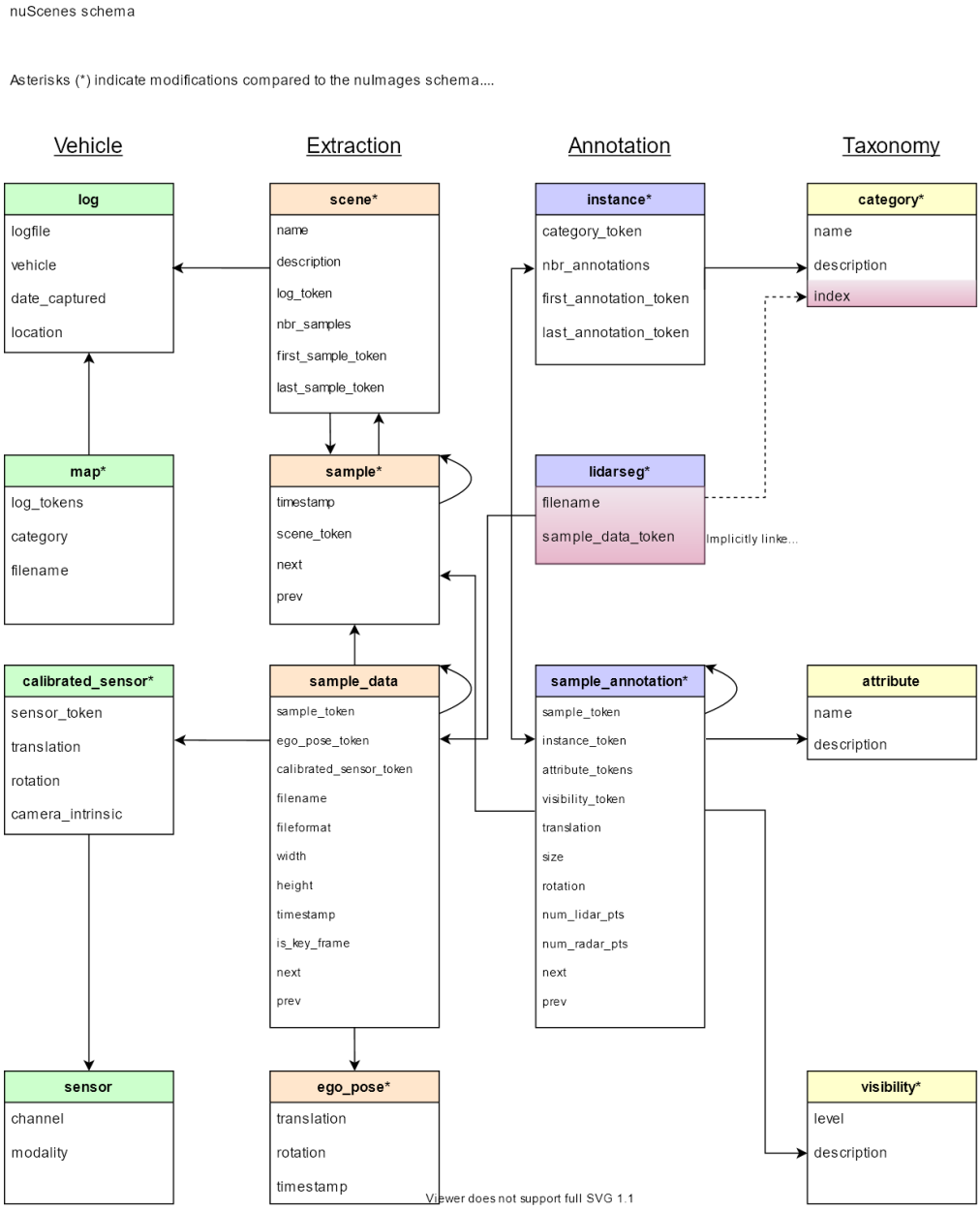
不同传感器的采样频率并不相同。nuScenes 中 Camera 采样频率约为 12Hz，LiDAR 约为 20Hz，Radar 约为 13Hz，因此不同传感器通常不会在完全相同的时间采集数据。为了提高不同传感器之间的数据对应关系，nuScenes 对 Camera 曝光和 LiDAR 扫描进行了时间同步，使 Camera 曝光时刻尽可能接近 LiDAR 扫描经过 Camera 视场的时刻。需要注意的是，即使进行了同步，不同传感器数据之间仍然可能存在一定时间差。在车辆运动过程中，即使很短的时间差，也会导致车辆位置发生变化。因此，在进行高精度 LiDAR 到 Camera 投影时，仅仅使用传感器之间固定的外参还不够，还需要考虑两个传感器采集时刻对应的车辆位姿。这个问题会在后续坐标转换实验中进一步介绍。

3.3. 数据集格式

nuScenes 的数据组织方式与常见的“images 文件夹+labels 文件夹”不同。它采用类似关系数据库的方式管理 Camera、LiDAR、Radar、车辆位姿、传感器标定和目标标注等信息。不同数据表之间通过 token 进行关联。token 可以理解为每一条数据记录的唯一编号。例如，一个 sample 具有自己的 sample token；通过这个 sample 可以找到对应的 Camera 和 LiDAR sample_data；再通过 sample_data 可以找到传感器标定信息和当前时刻的车辆位姿。

对于课程实践而言，并不需要一开始掌握 nuScenes 中的所有数据表。首先理解以下几个最常用的数据表即可。scene 用于描述约 20s 的连续驾驶片段，其中记录了场景名称、描述信息，以及第一个和最后一个 sample。sample 用于描述一个带标注关键时刻，其中包含时间戳以及该时刻对应的传感器数据和目标标注。sample_data 用于描述某一个具体传感器的一帧数据，其中包含真实数据文件路径、采集时间戳、传感器标定信息索引以及车辆位姿索引。calibrated_sensor 保存传感器标定信息，其中 translation 和 rotation 描述 Sensor 坐标系与 Ego 车辆坐标系之间的关系。对于 Camera，还会保存 camera_intrinsic，即相机内参矩阵。ego_pose 保存车辆在某一时刻的位置和姿态，可以理解为 Ego 车辆坐标系与 Global 全局坐标系之间的关系。sample_annotation 保存当前 sample 中的 3D 目标标注，包括目标的位置、尺寸、方向、类别和属性等信息。

因此，在后续实验中可以重点记住两组关系。第一组是数据组织关系：scene→sample→sample_data，它解决的是“当前正在读取哪一段驾驶数据、哪个关键时刻、哪一个传感器文件”的问题。第二组是坐标转换关系：Sensor→Ego→Global，calibrated_sensor 负责 Sensor 与 Ego 之间的转换，ego_pose 负责 Ego 与 Global 之间的转换。后续将 LiDAR 点云投影到 Camera 图像时，还需要执行反向转换，将 LiDAR 点从 LiDAR 坐标系转换到 Global 坐标系，再转换到 Camera 采集时刻的 Ego 坐标系和 Camera 坐标系，最终投影到二维图像平面。



3. 4. 数据集标注

nuScenes 完成原始传感器数据采集后，并不会对 Camera、LiDAR 和 Radar 的每一帧数据全部进行人工标注，而是按照约 2Hz 的频率选择关键帧进行标注，也就是大约每 0.5s 标注一次。在 nuScenes 中，一个这样的带标注关键时刻称为 sample。因此，需要重点区分 scene 和 sample 两个概念。scene 表示约 20s 的连续驾驶片段，而 sample 表示 scene 中的某一个带标注关键时刻。由于标注频率约为 2Hz，因此一个 20s 的 scene 通常包含约 40 个 sample。

nuScenes 还对关键帧中的道路目标进行人工标注。目标标注主要以 3D Bounding Box 形式给出，其中包含目标类别、三维位置、尺寸、方向以及相关属性。需要特别注意的是，nuScenes 主要提供的是三维目标框，而不是直接提供 YOLO 训练所使用的二维目标框。因此，在本课程后续的 YOLO 实践中，需要利用 Camera 标定参数将 3D Bounding Box 投影到图像平面，从而得到二维目标框。

除了带标注的关键帧之外，nuScenes 还保存了关键帧之间采集的部分高频传感器数据。关键帧数据主要存放在 samples 目录中，而中间传感器数据主要存放在 sweeps 目录中。对于本课程的基础实践，主要使用 samples 中的关键帧数据即可；在进行多帧 LiDAR 融合等进一步实验时，才需要使用 sweeps 数据。

3. 5. 数据集下载

本课程提供两种数据下载方式：一是通过 nuScenes 官方网站下载原始数据，二是通过课程提供的夸克云盘下载已经整理好的教学数据。对于第一次使用 nuScenes 的学生，建议首先使用课程提供的数据完成实验。课程数据删除了后续实验不会使用的大量 sweeps 中间帧，因此占用空间更小、下载速度更快。需要进一步开展多帧 LiDAR 融合、多帧 Radar 融合等实验时，可以再从 nuScenes 官方网站下载完整数据。

nuScenes 官方下载地址为：https://www.nuscenes.org/download。下载数据前需要注册 nuScenes 账号并登录，同时同意官方数据使用协议。nuScenes 官方 DevKit 说明中建议将下载的数据统一解压到同一个数据根目录，例如：/data/sets/nuscenes/。不要将每个压缩包分别解压到不同目录。因为 nuScenes 的多个压缩包实际上是同一个数据集的不同组成部分，解压后需要共同组成一个完整的数据目录。

对于初次学习和程序调试，可以首先下载 v1.0-mini。官方 Tutorial 直接使用 v1.0-mini 进行数据集介绍和 DevKit 演示。v1.0-mini 是一个已经打包完成的小型数据集，下载文件为：v1.0-mini.tgz。该数据集只包含 10 个 scene，适合检查环境安装是否正确，以及学习 scene、sample、sample_data 等基本数据结构。

考虑到 nuScenes 完整数据集文件较大，同时官方下载可能受到账号注册、网络速度以及存储空间的影响，本课程另外提供已经整理好的 nuScenes 教学数据。课程数据下载地址：https://pan.quark.cn/s/2c9a1ed5a9bd。学生可以使用夸克网盘客户端保存并下载数据。课程云盘中的数据主要用于本课程实验，因此与 nuScenes 官方下载的完整数据相比进行了精简，最重要的区别是课程云盘数据不提供 sweeps 数据。

无论通过 nuScenes 官方网站还是课程云盘下载数据，最终都建议将数据整理到同一个 nuScenes 数据根目录下。完整 nuScenes 数据集主要由地图数据、关键帧传感器数据、中间帧传感器数据以及标注和元数据 4 部分组成。完整目录结构如下：

```
nuscenes/
│
├──maps/                        # 地图数据，用于保存不同采集区域的地图及相关信息
│
├──samples/                     # 关键帧传感器数据，与约2Hz的标注sample对应
│   ├──CAM_FRONT/               # 前视Camera关键帧图像
│   ├──LIDAR_TOP/               # 车顶LiDAR关键帧点云
│   ├──RADAR_FRONT/             # 前方Radar关键帧数据
│   └──...                      # 其他Camera和Radar传感器通道
│
├──sweeps/                      # 关键帧之间的中间传感器数据，用于多帧融合和时序任务
│   └──...                      # 内部同样按照Camera、LiDAR和Radar传感器通道组织
│
└──v1.0-trainval/               # 数据集标注、索引、标定参数和车辆位姿等元数据
    ├──attribute.json            # 目标属性，例如moving、parked、stopped等状态
    ├──calibrated_sensor.json    # 传感器标定参数，包括Sensor到Ego的外参及Camera内参
    ├──category.json             # 目标语义类别定义
    ├──ego_pose.json             # 不同传感器采集时刻车辆在Global坐标系中的位置和姿态
    ├──instance.json             # 同一scene中目标实例的连续身份信息
    ├──log.json                  # 原始驾驶日志信息，包括采集地点等
    ├──map.json                  # scene/log与地图文件之间的对应关系
    ├──sample.json               # 约2Hz的标注关键帧及其前后时序关系
    ├──sample_annotation.json    # 每个sample中的3D目标框及其类别、尺寸、位置和方向等标注
    ├──sample_data.json          # 具体Camera、LiDAR和Radar文件及其时间戳、标定和位姿索引
    ├──scene.json                # 约20s连续驾驶场景及其首尾sample信息
    ├──sensor.json               # Camera、LiDAR、Radar等传感器类型及通道定义
    └──visibility.json           # 目标在Camera图像中的可见程度定义
```

其中，samples 目录保存课程实践中最常使用的关键帧数据，包括 Camera 图像、LiDAR 点云和 Radar 数据。一个 sample 表示约 2Hz 的带标注关键时刻，而 samples 目录中的文件就是与这些关键时刻对应的实际传感器数据。sweeps 目录保存相邻关键帧之间采集的中间传感器数据。其内部同样按照 Camera、LiDAR 和 Radar 传感器通道组织。这些数据主要用于多帧 LiDAR 融合、多帧 Radar 融合以及时序感知等任务。

v1.0-trainval 目录保存数据集的标注和元数据。该目录中的 JSON 文件不存放 Camera 图像或 LiDAR 点云本身，而是负责描述不同数据之间的关系。例如，sample.json 描述当前关键帧，sample_data.json 指出该关键帧对应的具体传感器文件，calibrated_sensor.json 提供传感器标定参数，ego_pose.json 提供车辆位姿，sample_annotation.json 保存目标的 3D 标注。

从实际使用角度看，可以将 nuScenes 的数据组织方式简单理解为：samples 和 sweeps 负责保存真实的传感器数据，而 v1.0-trainval 中的 JSON 文件负责保存数据索引、传感器标定、车辆位姿和目标标注等信息。

3.6. nuscenes-devkit 使用

3.6.1. nuScenes-devkit 安装

nuScenes 官方提供了 Python 开发工具包 nuScenes-devkit，用于读取数据集中的 JSON 元数据、访问 Camera、LiDAR 和 Radar 文件、查询传感器标定和车辆位姿、读取目标标注以及完成常用的数据可视化。后续实验中的 Camera 图像读取、LiDAR 点云显示、LiDAR 到 Camera 投影以及 3D 目标框读取等操作，都可以通过 nuScenes-devkit 完成。nuScenes-devkit 官方 GitHub 地址为：https://github.com/nutonomy/nuscenes-devkit。

目前官方 DevKit 支持 Python3.9 和 Python3.12，并推荐使用 Python3.12。对于本课程中的基础数据读取和可视化实验，直接通过 pip 安装即可；如果希望查看和修改 DevKit 源码，或者使用 Prediction、Tracking 等完整功能，则建议下载 GitHub 源码。官方特别说明，pip 发布包并不包含 Prediction 和 Tracking 所需的全部代码和依赖。

```
conda create -n nuscenes python=3.12 -y
conda activate nuscenes
pip install nuscenes-devkit
```

3.6.2. 数据集初始化与基本读取

使用 nuScenes-devkit 时，首先需要创建 NuScenes 对象。初始化后，DevKit 会自动读取 v1.0-trainval 中的 JSON 数据表。之后可以通过 scene 获取一个连续驾驶场景，再从 scene 中找到对应的 sample。在实际使用中，只需要重点理解 scene→sample→sample_data 这一基本关系。nusc.get()是最常用的查询函数，用于根据 token 读取对应的数据记录。

下面这部分代码完成了最基本的数据访问流程：首先选择一个 scene，然后获得其中一个 sample，最后从 sample 中找到具体的 Camera 和 LiDAR 数据。

```
from nuscenes.nuscenes import NuScenes
DATA_ROOT = "/path/to/nuscenes"
# 1.初始化数据集
nusc = NuScenes(
    version="v1.0-trainval",
    dataroot=DATA_ROOT,
    verbose=True
)
# 2.读取第一个 scene
```

```
scene = nusc.scene[0]
print("scene 名称：", scene["name"])
print("scene 描述：", scene["description"])
# 3.读取该 scene 的第一个 sample
sample = nusc.get(
    "sample",
    scene["first_sample_token"]
)
print("sample token：", sample["token"])
# 4.查看当前 sample 包含的传感器
print("传感器：")
for sensor_name in sample["data"]:
    print(sensor_name)
# 5.获取前视 Camera 和 LiDAR 的 token
cam_token = sample["data"]["CAM_FRONT"]
lidar_token = sample["data"]["LIDAR_TOP"]
print("CAM_FRONT token：", cam_token)
print("LIDAR_TOP token：", lidar_token)
# 6.获取实际数据文件路径
cam_path = nusc.get_sample_data_path(cam_token)
lidar_path = nusc.get_sample_data_path(lidar_token)
print("Camera 文件：", cam_path)
print("LiDAR 文件：", lidar_path)
```

3.6.3. 标定、车辆位姿与坐标转换

nuScenes 中的 3D 目标标注主要位于 Global 坐标系，而 Camera 和 LiDAR 数据分别位于各自的 Sensor 坐标系。因此，进行 LiDAR-Camera 投影或者 3D 标注投影时，需要使用传感器标定参数和车辆位姿。

calibrated_sensor 保存 Sensor 与 Ego 车辆坐标系之间的关系，其中 translation 和 rotation 为传感器外参；对于 Camera，还包含 camera_intrinsic 相机内参。ego_pose 保存传感器采集时刻 Ego 车辆在 Global 坐标系中的位置和姿态。因此后续常见的坐标关系为：Sensor → Ego → Global，或者反向：Global → Ego → Sensor。

DevKit 中的 Box 对象提供了 translate()和 rotate()函数，可以用于 3D 目标框的平移和旋转。官方内部实现同样通过这两个操作将 Global 坐标系中的 box 先转换到 Ego 坐标系，再转换到 Sensor 坐标系。

```
import numpy as np
from pyquaternion import Quaternion
#  获取 Camera sample_data
cam_data = nusc.get(
    "sample_data",
    cam_token
)
#  获取 Camera 标定参数
cam_calib = nusc.get(
    "calibrated_sensor",
```

```
        cam_data["calibrated_sensor_token"]
)
#  获取 Camera 采集时刻车辆位姿
cam_pose = nusc.get(
        "ego_pose",
        cam_data["ego_pose_token"]
)
print("Camera 外参平移：")
print(cam_calib["translation"])
print("Camera 外参旋转：")
print(cam_calib["rotation"])
print("Camera 内参：")
print(cam_calib["camera_intrinsic"])
print("Ego 位置：")
print(cam_pose["translation"])
print("Ego 姿态：")
print(cam_pose["rotation"])
#  获取一个 3D Box
box = nusc.get_box(
        sample["anns"][0]
)
# Box 支持平移和旋转
box.translate(
        -np.array(cam_pose["translation"])
)
box.rotate(
        Quaternion(cam_pose["rotation"]).inverse
)
box.translate(
        -np.array(cam_calib["translation"])
)
box.rotate(
        Quaternion(cam_calib["rotation"]).inverse
)
```

上面的手动变换主要用于帮助理解坐标转换过程。实际编程时，很多情况下不需要自己重复实现这些操作，因为 nuScenes 已经提供了更高级的封装函数。例如：

```
data_path, boxes, camera_intrinsic = nusc.get_sample_data(
        cam_token
)
```

get_sample_data()会读取当前传感器的数据，并将相关 3D Bounding Box 自动转换到当前 Sensor 坐标系。对于 CAM_FRONT 而言，返回的 boxes 已经处于 CAM_FRONT 坐标系，可以直接用于后续 Camera 投影。官方源码明确说明该函数返回的 boxes 会被转换到当前 sensor coordinate frame。因此，在后续 3D 转 2D 实验中，通常优先使用 get_sample_data()，

而不需要每次手动完成 Global→Ego→Camera 的变换。

3.6.4. 投影与可视化相关接口

nuScenes-devkit 已经封装了多种投影和可视化函数。这些函数一方面可以直接用于结果展示，另一方面也可以帮助理解后续 3D 转 2D 数据处理的内部过程。render_sample_data()可以显示某一个 Camera、LiDAR 或 Radar 的 sample_data；Camera 数据可以同时显示目标标注。官方 Tutorial 直接使用该函数查看 CAM_FRONT 等传感器数据。

map_pointcloud_to_image()可以完成 LiDAR 点云到 Camera 图像的完整坐标转换。该函数内部会读取 LiDAR 和 Camera 各自的 calibrated_sensor 和 ego_pose，完成 Point Sensor→Ego→Global→Ego→Camera 的变换，并最终投影到图像平面。

render_pointcloud_in_image()在 map_pointcloud_to_image()基础上进一步完成可视化，可直接将 LiDAR 点绘制到 Camera 图像中。

对于 3D 目标框，box.corners()可以获取 3D 框的 8 个角点，view_points()可以使用 Camera 内参将 Camera 坐标系中的三维点投影到二维图像平面。官方 geometry_utils.py 中将 view_points()定义为将 3D 点映射到 2D 平面的通用投影函数。

常用接口示例如下：

```
from nuscenes.utils.geometry_utils import (
    view_points,
    box_in_image,
    BoxVisibility
)
# 1.Camera 数据及 3D 标注可视化
nusc.render_sample_data(
    cam_token
)
# 2.LiDAR 数据可视化
# 课程数据不包含 sweeps，因此使用 nsweeps=1
nusc.render_sample_data(
    lidar_token,
    nsweeps=1
)
# 3.LiDAR 点云投影到 Camera
points, depth, image = nusc.explorer.map_pointcloud_to_image(
    lidar_token,
    cam_token,
    nsweeps=1
)
# 4.直接显示 LiDAR-Camera 投影结果
nusc.render_pointcloud_in_image(
    sample_token=sample["token"],
    pointsensor_channel="LIDAR_TOP",
    camera_channel="CAM_FRONT",
    nsweeps=1
)
# 5.获取 Camera 坐标系下的 3D Box 和 Camera 内参
```

```
image_path, boxes, camera_intrinsic = nusc.get_sample_data(
    cam_token
)
# 6.获取 3D Bounding Box 角点
box = boxes[0]
corners_3d = box.corners()
# 7.将 Camera 坐标系中的 3D 点投影到图像
corners_2d = view_points(
    corners_3d,
    camera_intrinsic,
    normalize=True
)
# 8.判断 3D Box 是否位于 Camera 视野内
visible = box_in_image(
    box,
    camera_intrinsic,
    (1600, 900),
    vis_level=BoxVisibility.ANY
)
print("目标是否可见：", visible)
```

3. 7. 2D 目标检测标签转换

nuScenes 主要面向自动驾驶三维感知任务，目标标注以三维包围框（3D Bounding Box）为主，而 Ultralytics YOLO 等常用二维目标检测网络需要图像平面上的二维矩形框。因此，在使用 nuScenes 开展 YOLO 二维目标检测实验之前，需要将三维目标标注变换到相机坐标系、投影到图像平面，并进一步转换为 YOLO 标签格式。

本课程提供完整 nuScenes 转换 YOLO 的程序以及预先转换的数据集。学生可以直接使用课程数据训练模型，也可以自行运行转换程序，理解坐标变换、相机投影、目标筛选、数据降采样和训练集划分的完整流程。需要特别说明的是，本课程生成的二维框是三维框在图像平面上的投影包围框，并不是人工沿目标可见轮廓标注的紧致二维框。因此，在遮挡、截断、近距离目标或细长目标上，投影框与常规二维检测数据集中的人工框可能存在差异。本数据集适合用于学习自动驾驶数据处理及二维检测流程，但不应将其视为严格的人工二维标注基准。

转换代码包链接：https://pan.quark.cn/s/848ce1f1ac5f

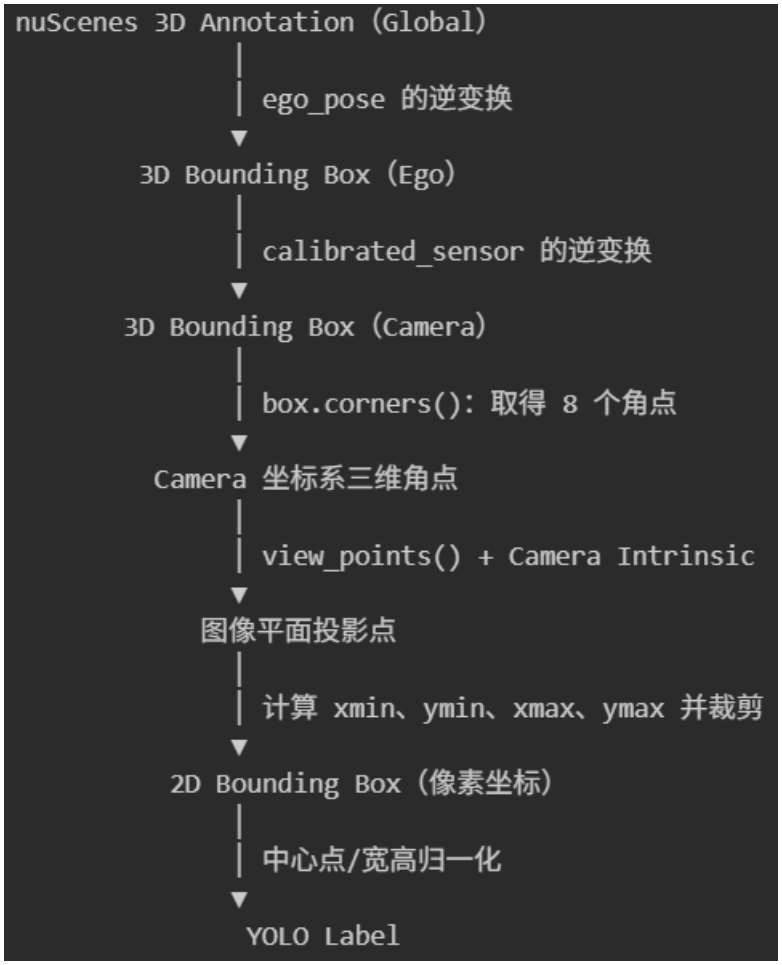
转换后的数据集链接：https://pan.quark.cn/s/b7135c24bca3

3. 7. 1. 转换原理

nuScenes 中的 3D Bounding Box 首先定义在 Global 坐标系中，而 YOLO 需要的是 Camera 图像坐标系中的二维矩形框。因此，转换过程本质上包括 3D 坐标转换和 Camera 投影两个步骤。

首先，根据目标标注、车辆位姿和传感器标定参数，将 Global 坐标系中的 3D Bounding Box 转换到指定 Camera 坐标系。nuScenes-devkit 已经在 get_sample_data()等函数中封装了相应的坐标转换操作，因此实际编程时通常不需要手工重复实现 Global→Ego→Camera 的完整变换。得到 Camera 坐标系中的 3D Bounding Box 后，可以通过 box.corners()获取 3D 框的 8 个角点，再利用 Camera 内参和 view_points()函数将这些三维角点投影到图像平面。

最后，根据所有有效投影点的最小和最大像素坐标计算二维包围框。整个转换流程可以概括为：



3.7.2. 目标类别

nuScenes 原始标注包含更细的类别层级，例如不同类型的行人和公交车。本课程根据 nuScenes Detection 任务的类别映射关系将其归并为 10 类。类别 ID 由转换程序中的 CLASS_NAMES 顺序确定，课程当前版本如下：

类别 ID	类别名称	典型原始类别映射
0	car	vehicle.car
1	truck	vehicle.truck
2	bus	vehicle.bus.bendy、vehicle.bus.rigid
3	trailer	vehicle.trailer
4	construction_vehicle	vehicle.construction
5	pedestrian	adult、child、construction_worker、police_officer 等行人子类
6	motorcycle	vehicle.motorcycle
7	bicycle	vehicle.bicycle
8	traffic_cone	movable_object.trafficcone
9	barrier	movable_object.barrier

3.7.3. 数据集划分与转换脚本

本课程不是把全部图像随机混合后再划分，而是先按照 location 生成四个互相独立的数

据集： boston-seaport、singapore-hollandvillage、singapore-onenorth 和 singapore-queenstown。每个 location 内再以 scene 为最小单位，使用固定随机种子 42 划分 train、val、test，比例为 7:1:2。同一个 scene 的连续帧不会同时进入不同 split，从而降低相邻画面泄漏导致评估结果虚高的风险。由于一个 scene 包含的有效帧数不同，最终图像数量比例会接近但不一定严格等于 70%、10%和 20%。

Location	Train	Val	Test	合计
boston-seaport	9,420	1,402	2,709	13,531
singapore-hollandvillage	365	56	88	509
singapore-onenorth	3,634	461	1,015	5,110
singapore-queenstown	1,661	205	486	2,352
总计	**15,080**	**2,124**	**4,298**	**21,502**

转换程序位于课程代码仓库的 nuscenes_yolo_converter 目录。程序依赖通过 pip 安装的 nuscenes-devkit，不依赖本地修改版 devkit。--dataroot 指向 nuScenes 根目录，该目录下应包含 samples/、sweeps/、maps/和 v1.0-trainval/等内容。--output 指定转换结果位置；--overwrite 会删除并重新创建非空输出目录，应确认目标路径无误后再使用。--image-mode copy 会生成独立图像副本，兼容性最好但占用额外磁盘空间；同一文件系统中也可尝试 hardlink 以节省空间，Linux 下可用 symlink，但移动数据集或改变源数据路径时需注意链接有效性。Linux 环境下可在项目根目录执行：

```
conda activate yolo
python nuscenes_yolo_converter/convert.py \
  --dataroot /data/nuscenes \
  --output nuscenes_yolo_converter/output/nuscenes_yolo_vis34 \
  --channels CAM_FRONT \
  --visibilities 3 4 \
  --exclude-night \
  --min-distance 1.0 \
  --train-ratio 0.7 \
  --val-ratio 0.1 \
  --seed 42 \
  --class-mode detection10 \
  --image-mode copy \
  --overwrite
```

转换程序为每个 location 生成一个可以独立移动和使用的 Ultralytics YOLO 数据集。images/与 labels/下的图像和标签通过文件主名一一对应；data.yaml 定义相对数据路径和类别名称，可以直接传给 Ultralytics YOLO；classes.txt 保存按 ID 排列的类别名称；manifest.csv 记录 split、location、源图像、输出图像和标签数量，便于追溯；conversion_summary.json 保存转换参数及统计信息。数据集内部使用相对路径，因此整个 location 目录移动到其他机器后无需修改内部清单。manifest.csv 中的 source_image 仅作为来源追溯信息记录原始绝对路径，不参与 YOLO 训练。以 boston-seaport 为例：

```
nuscenes_yolo_vis34/
├── conversion_summary.json
├── boston-seaport/
│   ├── data.yaml
│   ├── classes.txt
│   ├── conversion_summary.json
│   ├── manifest.csv
│   ├── manifests/
│   │   ├── train.txt
│   │   ├── val.txt
│   │   └── test.txt
│   ├── images/
│   │   ├── train/
│   │   ├── val/
│   │   └── test/
│   └── labels/
│       ├── train/
│       ├── val/
│       └── test/
├── singapore-hollandvillage/
├── singapore-onenorth/
└── singapore-queenstown/
```

3.7.4. 目标筛选与关键参数

nuScenes 的 visibility_token 描述目标在传感器视角中的可见程度，而不是目标是否处于 Camera 视场内。官方四档可见度的含义为：1 表示可见比例 0 ~ 40%，2 表示 40 ~ 60%，3 表示 60 ~ 80%，4 表示 80 ~ 100%。课程采用 3 4，即只保留估计可见比例不低于 60% 的目标。目标是否真正落入当前 Camera 图像，还要由相机坐标中的深度、中心位置和投影框范围继续判断。因此，--visibilities 3 4 与 Camera 视场筛选是两个先后配合但含义不同的条件。

选择 3、4 档是课程数据质量与样本数量之间的折中。若同时保留 1、2 档，可以获得更多困难样本，但严重遮挡目标的完整三维框投影往往覆盖遮挡物或大片背景，不等同于目标当前可见部分的紧致二维框；对于入门二维检测实验，这会增加标签噪声。后续若研究遮挡鲁棒性，可改为--visibilities 1 2 3 4 重新生成数据，并对结果进行可视化抽查。

本次课程数据在上述条件下从 30,162 张候选图像中删除 2,069 张无有效标注图像，并因 1 m 距离采样删除 6,591 张近重复图像，最终保留 21,502 张图像和 128,712 个二维目标框。

参数	课程采用值	作用与选择依据
`--channels`	`CAM_FRONT`	仅使用车辆正前方相机，便于开展单相机二维目标检测。需要多相机数据时可显式传入其他Camera通道。
`--visibilities`	`3 4`	只保留可见比例较高的目标，减少严重遮挡目标通过完整三维框投影后产生的大框、松框和歧义标签。
`--min-distance`	`1.0` m	同一场景、同一相机中，车辆从上一张保留帧的Ego平面位置移动至少1 m才保留新帧，以减少停车或低速行驶造成的近重复图像。
`--exclude-night`	开启	排除场景描述中带有`night`的场景，使课程基础实验集中在光照较稳定的日间图像上。
`--min-width`	10 px	去除投影宽度过小、训练价值有限且容易受量化误差影响的目标。
`--min-height`	10 px	去除投影高度过小的目标。
`--min-area`	100 px^2	对二维框面积设置下限，与宽高约束共同过滤极小框。
`require_center_inside`	开启	要求三维框中心的投影位于图像内，避免只在图像边缘露出很小部分、但投影框异常大的目标。
`require_all_corners_in_front`	开启	要求三维框8个角点都位于相机近平面之前，避免目标穿过或位于相机后方时产生不稳定投影。
`require_fully_inside`	关闭	不要求完整投影框都在图像内部。边缘截断目标仍可用于YOLO，程序会把框裁剪到图像边界。
`--train-ratio` / `--val-ratio`	`0.7` / `0.1`	训练、验证、测试比例为7:1:2，增加独立测试集比例，便于课程中的最终评估。
`--seed`	`42`	固定场景划分顺序，使不同学生和不同机器上的结果可复现。
`--class-mode`	`detection10`	使用nuScenes Detection任务的10个主要检测类别，而不是保留全部细粒度原始类别。

4. YOLO 配置及使用示例

代码链接：https://pan.quark.cn/s/792ccde19a3a

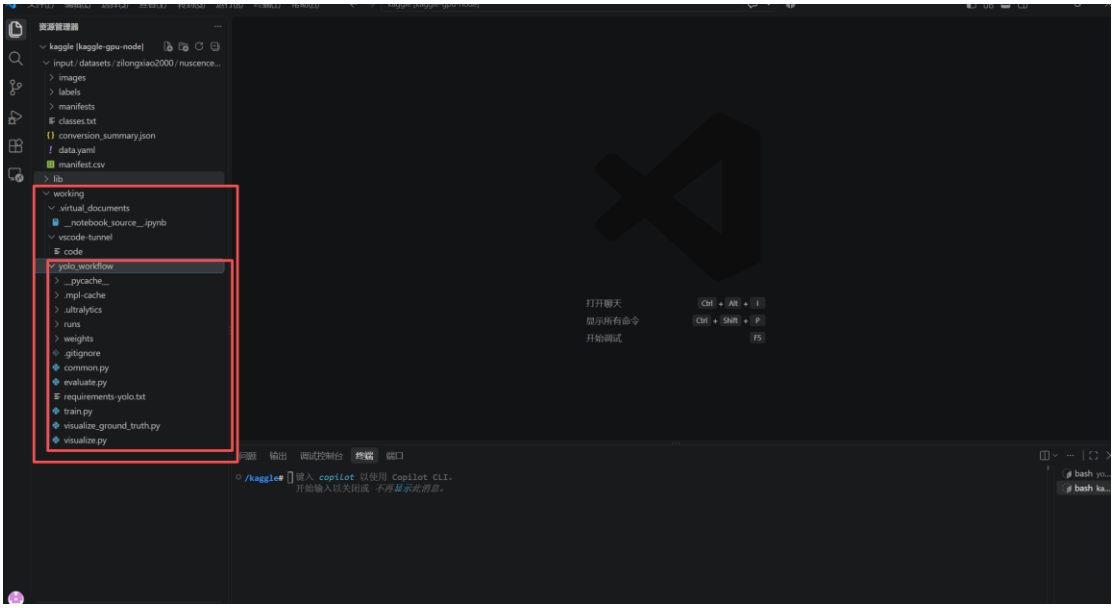
使用 3.7 下载的数据集，任意一个 location split 均可

YOLO 是 You Only Look Once 的缩写，是一种目标检测算法，YOLO 在进行目标检测任务时将输入图像划分为若干网格，每个网格负责预测目标中心落在该网格内的目标，直接输出目标的边界框坐标、置信度以及类别概率。其检测速度快，能够满足实时性要求较高的应用场景，而且可以同时检测多种不同类别的目标，对不同形状、大小和姿态的目标都有较好的检测效果。

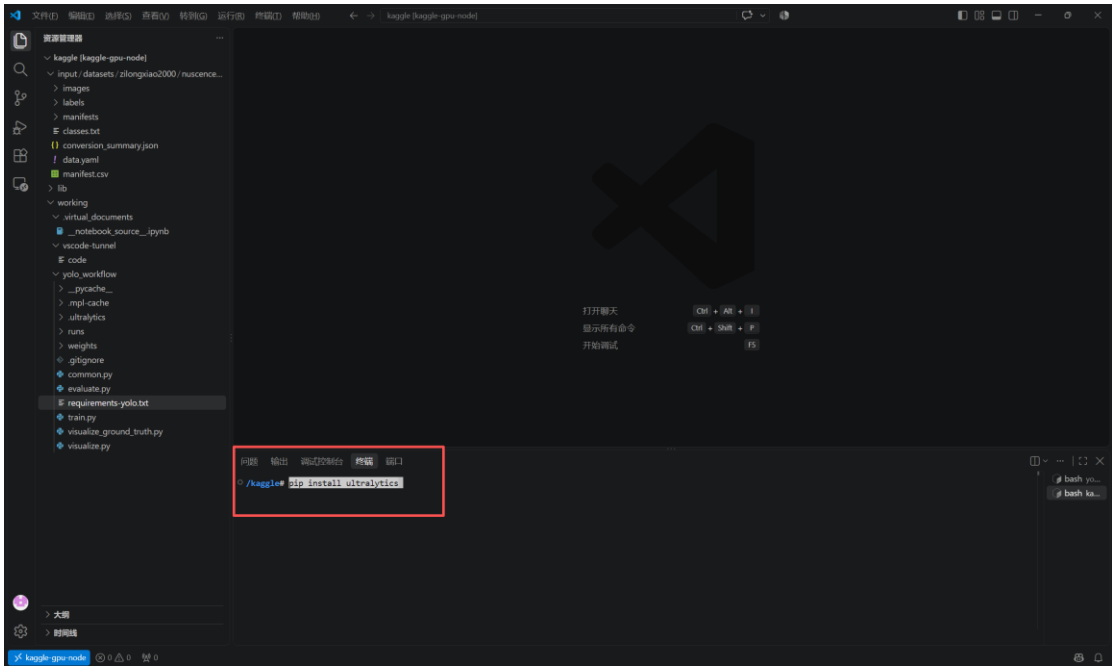
使用 YOLO 进行目标检测之前，我们需要准备数据集（自制或者下载）来训练模型，这样它才具备对某几类物体的目标检测能力，换句话说，我们需要先准备 n 张有 m 类物体的图像以及配套的标注文件，训练之后 YOLO 才具备对这 m 类物体的目标检测能力。

4. 1. 训练模型—免费 GPU 资源

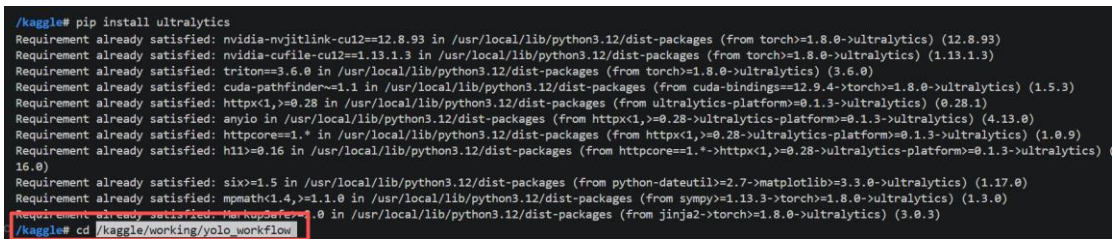
上传上述下载好的代码到 working 目录下



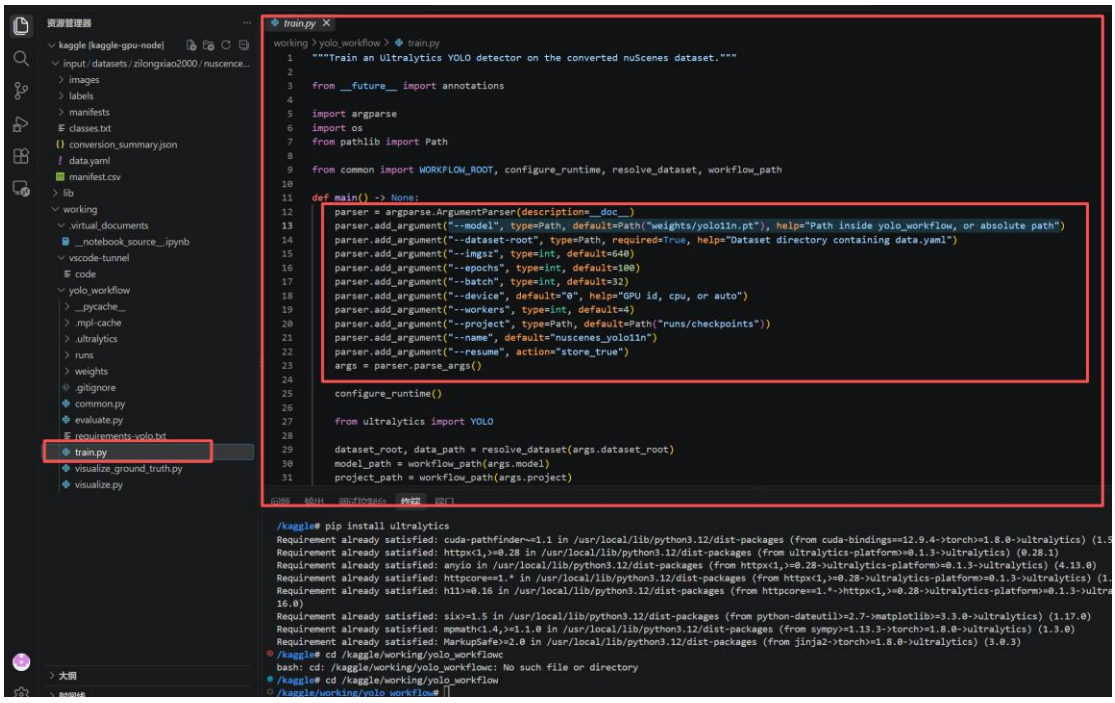
安装配置环境，在终端中输入 pip install ultralytics



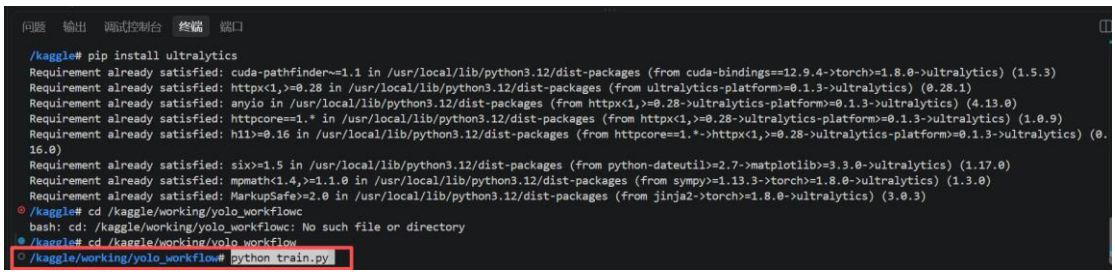
从根目录进入代码执行目录：cd /kaggle/working/yolo_workflow



找到训练文件 train.py，自行修改配置内容，包括模型、训练数据路径、图像尺寸、训练轮次、batch 等等



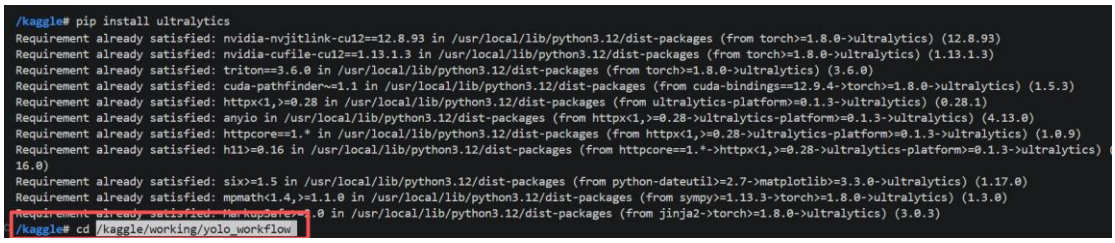
执行训练 python train.py



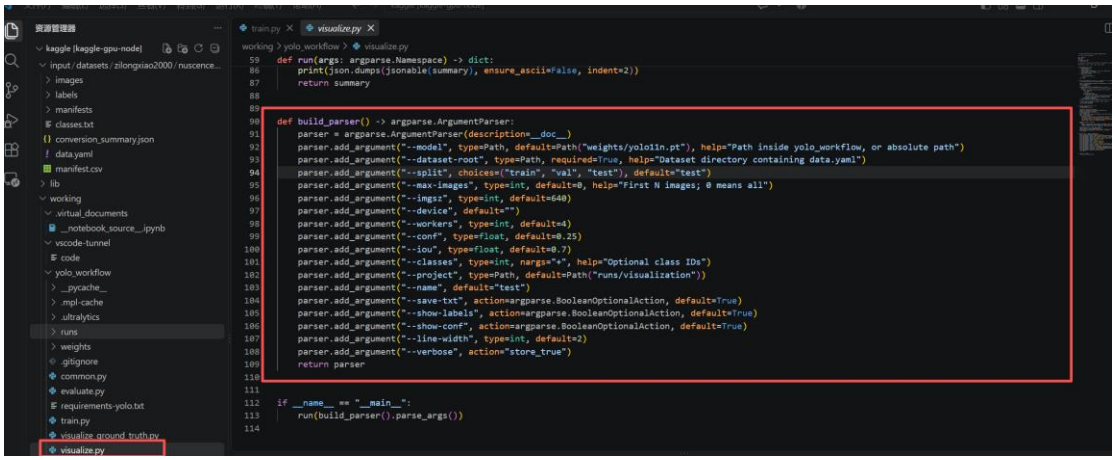
4. 2. 模型测试—免费 GPU 资源

数据集，环境，代码均和上述训练一致

从根目录进入代码执行目录：cd /kaggle/working/yolo_workflow



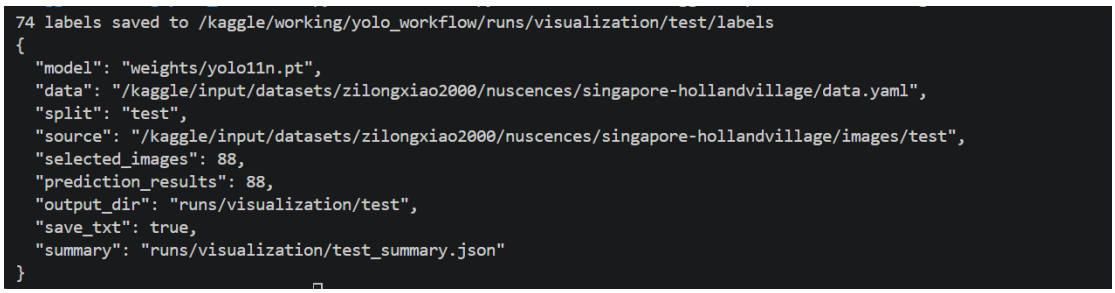
找到测试文件 visualize.py，自行修改配置内容，包括模型、训练数据路径、图像尺寸、训练轮次、batch 等等

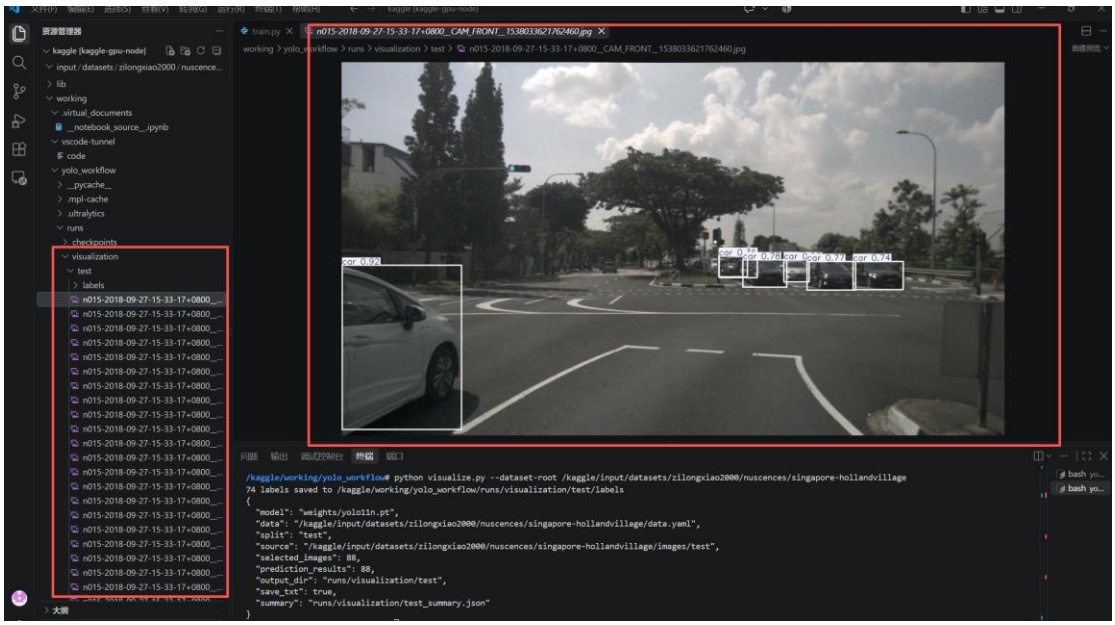


执行训练 python visualize.py



执行结果：

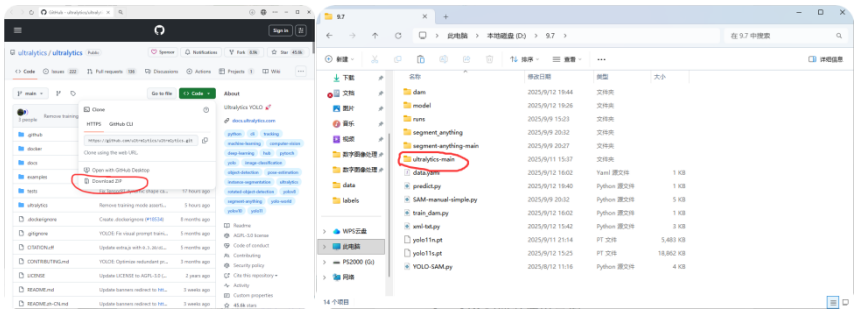




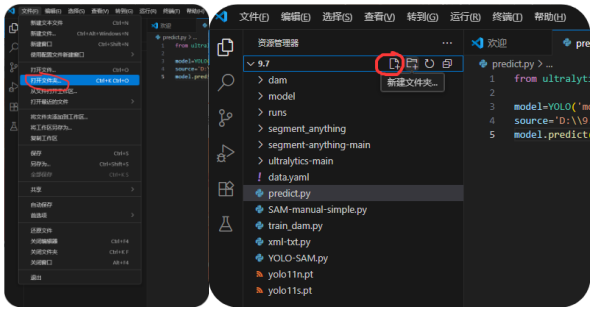
4. 3. 训练模型—本地

首先将之前安装了 pytorch 的环境激活，然后输入“pip install ultralytics”并回车，会自动下载 yolo 所需的所有库，有需要输入 y 或者 n 的输入 y 即可，另外还需要创建一个文件夹，并放入 ultralytics 的 main 代码

main 代码下载地址：https://github.com/ultralytics/ultralytics，点击下载，放入新创建的文件夹解压即可



以 Vscode 为例，打开 VS，Shift+Ctrl+P 选择安装了 pytorch 的虚拟环境，打开刚刚创建的文件夹，新建一个 python 文件（记得加后缀 py），新建一个 yaml 文件，内容如下，写完后一定要 Ctrl+S 保存





新建的 py 文件这样写即可，其中的路径都是相对路径，也可以更改，例如 yaml 文件中的为绝对路径，yolo11n 代表训练的是 yolo 的 11 版本的 n 大小的模型（模型大小 n<s<m<l<x，训练时长成倍增加），官网有用 coco 数据集预训练的模型（可检测 80 类物体），可以先下载，如果没有下载，在训练的过程中也会自动下载

简单介绍参数，epochs：总训练轮数，至少 20 轮，不高于 200 轮，轮数越多效果越好；

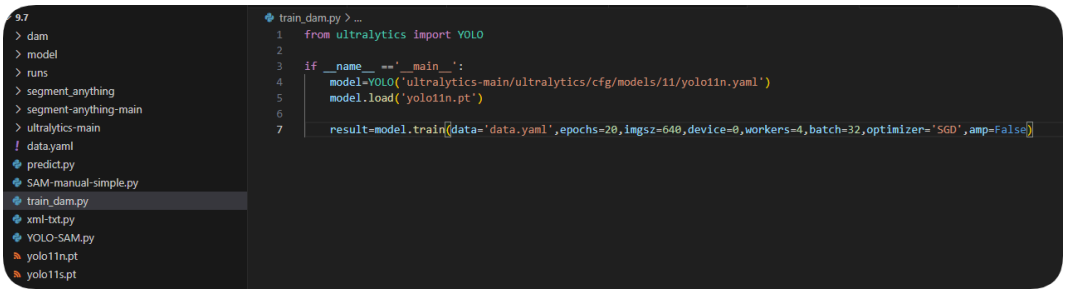
batch：批量大小，即单词输入多少图片训练；

imasz：训练图片尺寸（不用改）

workers：加载数据的工作线程数

device：指定训练的计算设备，无 N 卡则改为“device='cpu'”

optimizer='SGD'， #训练使用优化器，可选 auto,SGD,Adam,AdamW 等

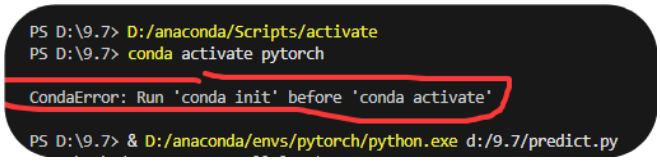


▼ Detection (COCO)

See Detection Docs for usage examples with these models trained on COCO, which includ

Model	size (pixels)	mAP^{val} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

点击运行即可，最开始可能会出现这样的提示，再点击一次运行就可以了出

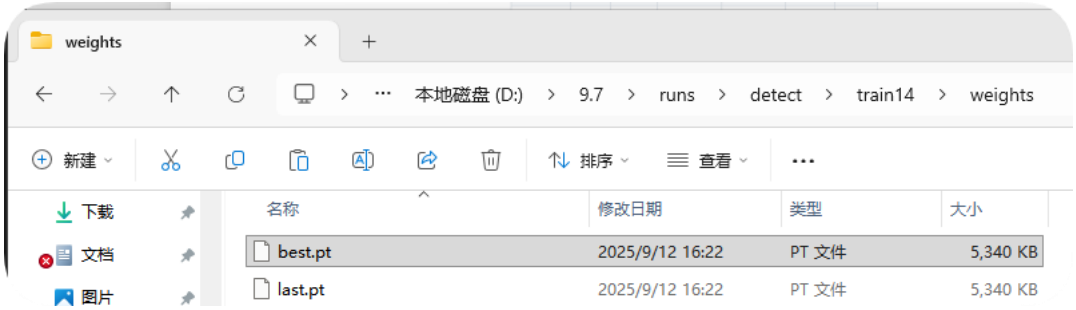


现这样的提示说明模型开始训练，等待它训练结束，会告诉你训练好的模型放在了哪里，best.pt 为训练好的模型

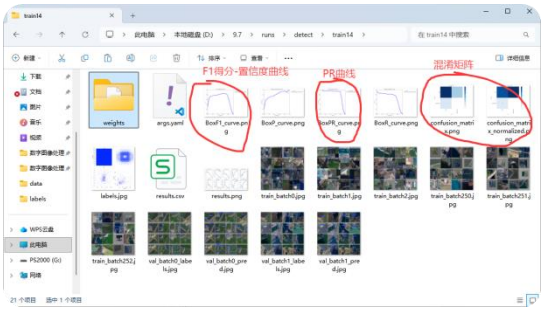
```
val: Scanning D:\9.7\dam\damcoco+GD\valid\labels.cache... 126 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 126/126  0.0s
Plotting labels to D:\9.7\runs\detect\train14\labels.jpg...
optimizer: SGD(lr=0.01, momentum=0.937) with parameter groups 81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bias(decay=0.0)
Image sizes 640 train, 640 val
Using 4 dataloader workers
Logging results to D:\9.7\runs\detect\train14
Starting training for 20 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       1/20      8.18G      2.065      3.624      2.116         57        640: 100% ━━━━━━━━━━━━ 25/25 0.4it/s 1:07
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 2.1it/s 1.0s
                   all        126        129    0.00333      0.977     0.0968     0.0453
```

```
Validating D:\9.7\runs\detect\train14\weights\best.pt...
Ultralytics 8.3.195  Python-3.9.23 torch-2.8.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060, 8151
YOLO11n summary (fused): 100 layers, 2,582,347 parameters, 0 gradients, 6.3 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95):
                   all        126        129      0.917      0.798      0.838      0.413
Speed: 0.5ms preprocess, 11.9ms inference, 0.0ms loss, 0.8ms postprocess per image
Results saved to D:\9.7\runs\detect\train14
```



在这个文件夹中还有一些图片可供大家评判模型的训练效果



4. 4. 模型预测—本地

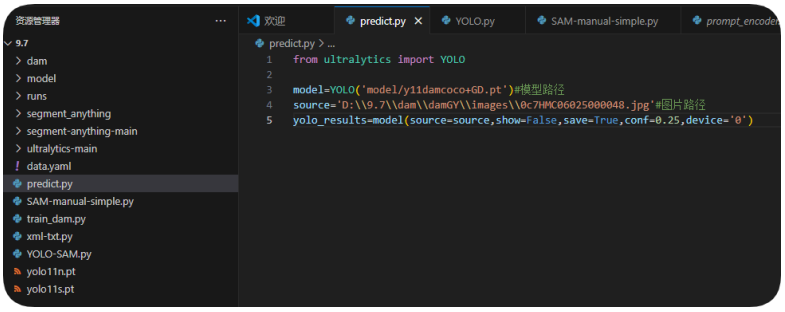
模型训练好之后，重新创建一个 py 文件，这样写就可以对一张图片进行预测，在图片中检测出所有经过训练的物体类别。

简单介绍参数，show：是否以窗口形式展示结果；

save：是否保存结果；

conf：最低置信度，低于这个置信度的检测目标将不会显示

device：指定的计算设备，无 N 卡则改为“device='cpu'”



同样的，会在命令行输出保存结果的保存路径

结果示例：

